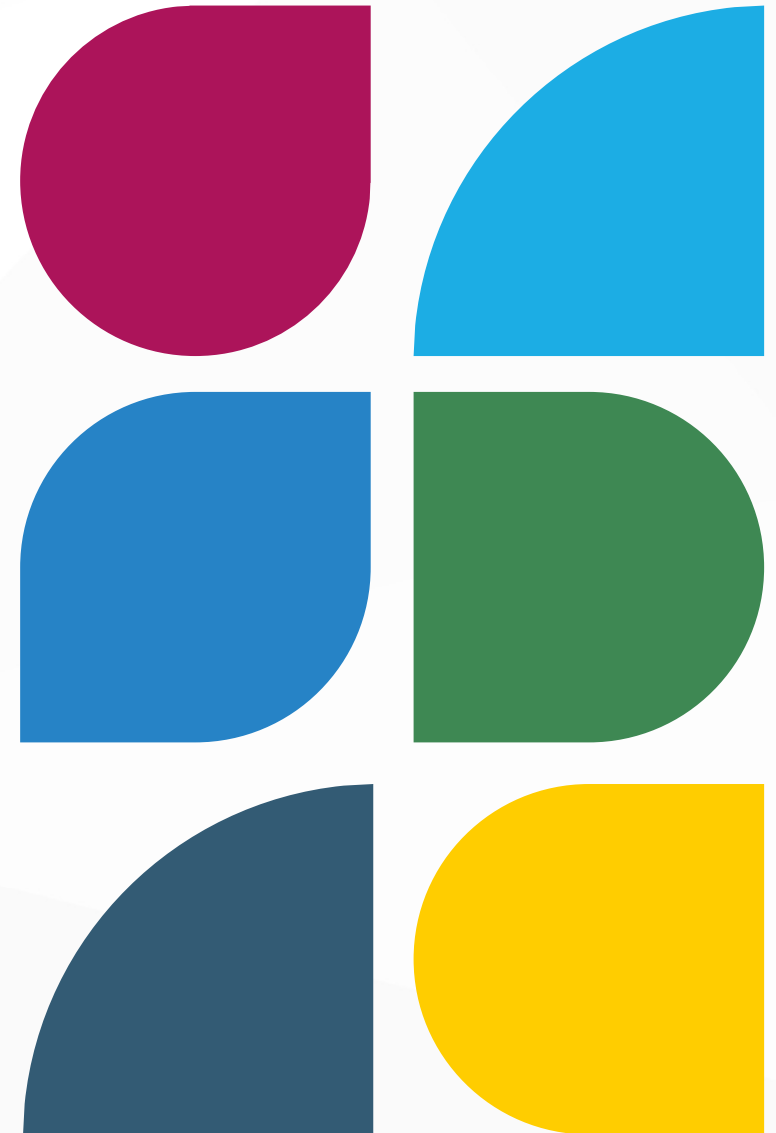
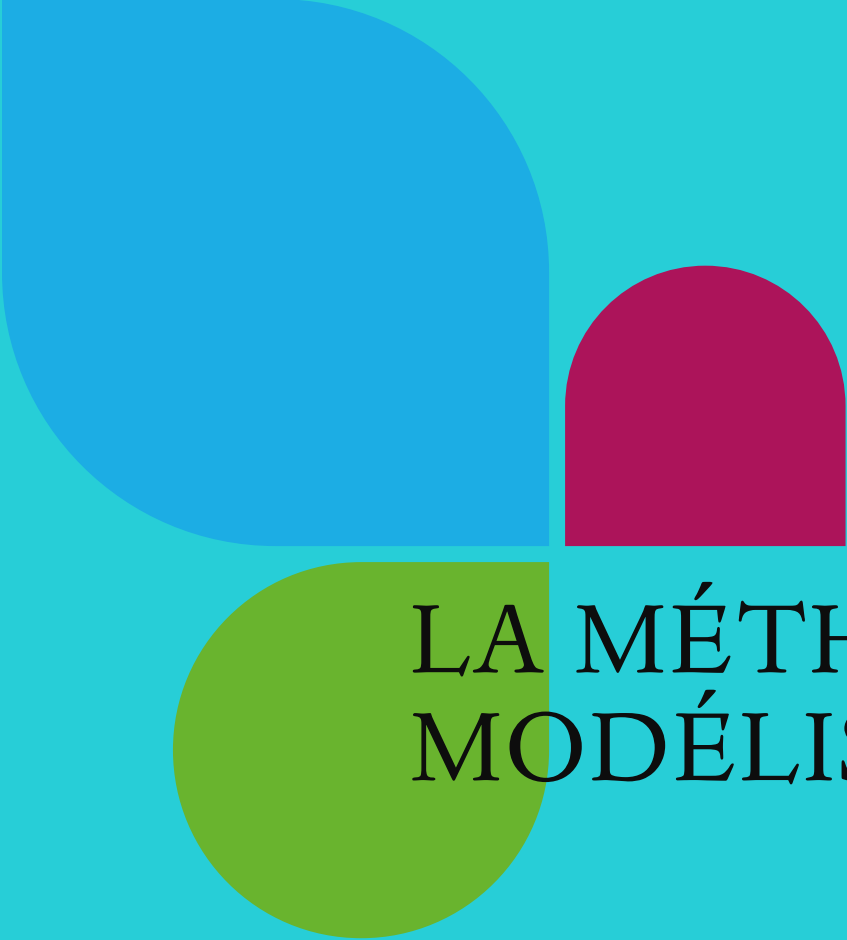


LA MÉTHODE MERISE

Concevoir et développer la persistance de données.
- Partie 1 -

Concepteur Développeur d'Applications





LA MÉTHODE MERISE MODÉLISATION

LA MÉTHODE MERISE

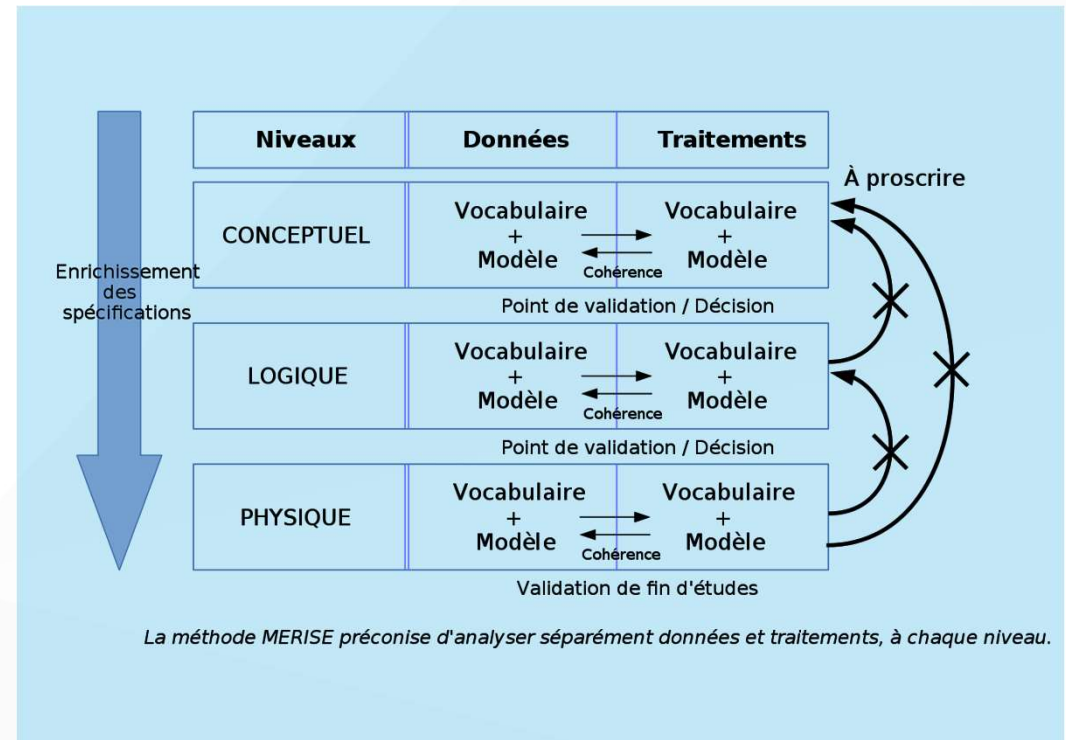
La méthode **MERISE** est une méthode d'analyse, de conception et de réalisation de systèmes d'informations, basée sur le principe de la séparation des données et des traitements.

- Généralement pour des projets de grande ampleur, de refonte d'un existant complexe, dans un environnement grand système.

La méthode **Merise** est le résultat de travaux datant des années 1970, à la suite d'une réflexion internationale.

- projet opérationnel au début entre 1972 et 1975 à la demande du ministère de l'industrie et a surtout été utilisée en France.
- Rendu célèbre en 1976, dans le monde entier par l'américain Peter Chen à la suite de la publication intitulée "[The Entity-Relationship Model](#)"

A la différence d'UML, son principe est de partir de l'analyse des données pour concevoir la future base de données. Tandis qu'UML part de l'analyse pour concevoir les objets (les classes) que nous allons manipuler.



Elle possède plusieurs modèles qui sont répartis sur 3 niveaux :

- Le niveau conceptuel,
- le niveau logique ou organisationnel,
- le niveau physique.

LES DIFFÉRENTS MODÈLES DE MERISE

Comme toute méthode, **MERISE** est riche de nombreux modèles, complété par des diagrammes.

Dans le cadre de la formation, nous allons surtout nous concentrer sur l'essence même de la méthode et par conséquent, nous ne pourrons pas étudier chaque modèle mais nous concentrer sur 3 modèles.

1. Modèle Conceptuel de Communication (MCC)
2. **Modèle Conceptuel des Données (MCD)**
3. Modèle Conceptuel des Traitements (MCT)
4. Modèle Organisationnel de Communication (MOC)
5. Modèle Organisationnel des Données (MOD)
6. Modèle Organisationnel des Traitements (MOT)
7. Modèle Logique de Communication (MLC)
8. **Modèle Logique des Données (MLD)**
9. Modèle Logique des Traitements (MLT)
10. Modèle Physique de Communication (MPC)
11. **Modèle Physique des Données (MPD)**
12. Modèle Physique des Traitements (MPT)

DÉMARCHE DE LA MÉTHODE

En schématisant la démarche de la méthode [Merise](#), on a alors 3 niveaux.

Nous allons donc surtout nous concentrer sur le dictionnaire des données, le MCD et le MPD dans le cadre de ce module.

1. Système d'information manuel
 - Expression des besoins
 - Cahier des charges
 - [Dictionnaire de données](#)
2. Modèle Conceptuel des données
 - Modèle logique
 - Modèle physique
3. Système d'information automatisé

NIVEAU CONCEPTUEL

L'étude conceptuelle **Merise** s'attache aux invariants de l'entreprise ou de l'organisme du point de vue du métier :

- quelles sont les activités ?
- les métiers gérés par l'entreprise ?
- quels sont les grands processus traités ?
- de quoi parle-t-on en matière de données ?
- quelles notions manipule-t-on ?...

Et ce indépendamment des choix techniques (comment fait-on ?) ou d'organisation (qui fait quoi ?) qui ne seront abordés que dans les niveaux suivants.

- **Le modèle conceptuel des données** ^{MCD}

- schéma représentant la structure du système d'information, du point de vue des données, c'est-à-dire les **dépendances** ou **relations entre les différentes données** du SI.

- **Le modèle conceptuel des traitements** ^{MCT}

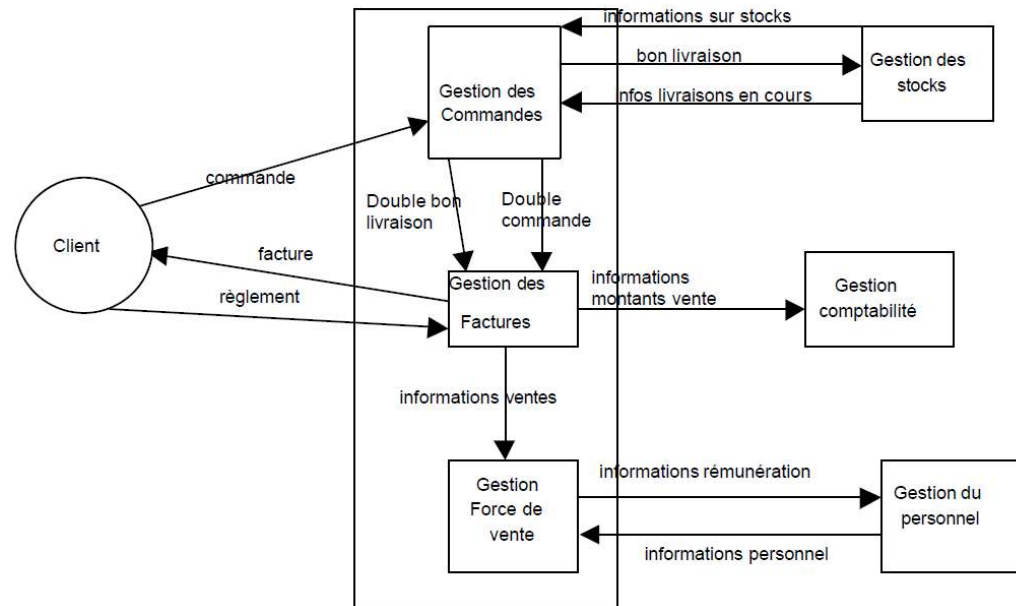
- schéma représentant les traitements, en réponse aux événements à traiter.

EXEMPLE

Diagramme Conceptuel des flux

- Il montre tous les flux du système en ne considérant qu'un seul acteur interne : l'entreprise.

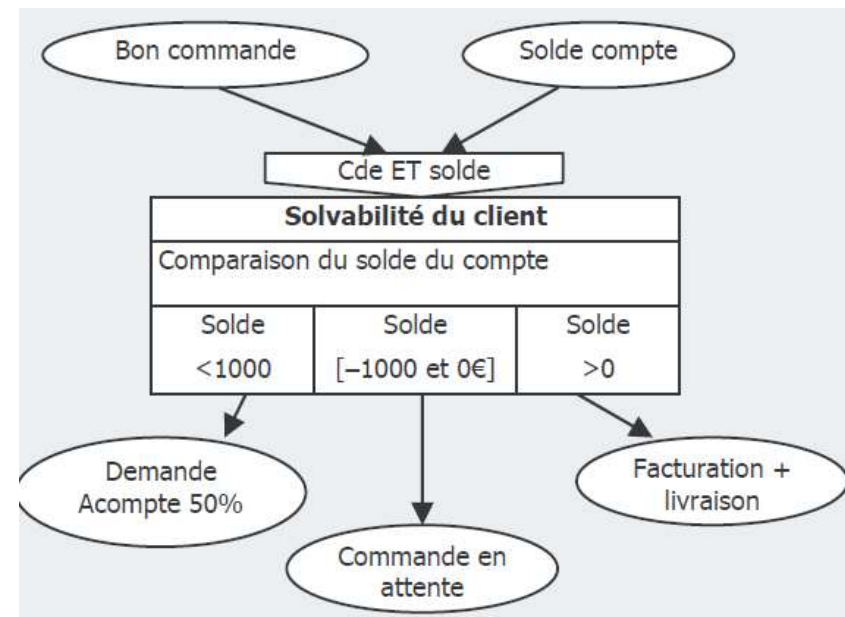
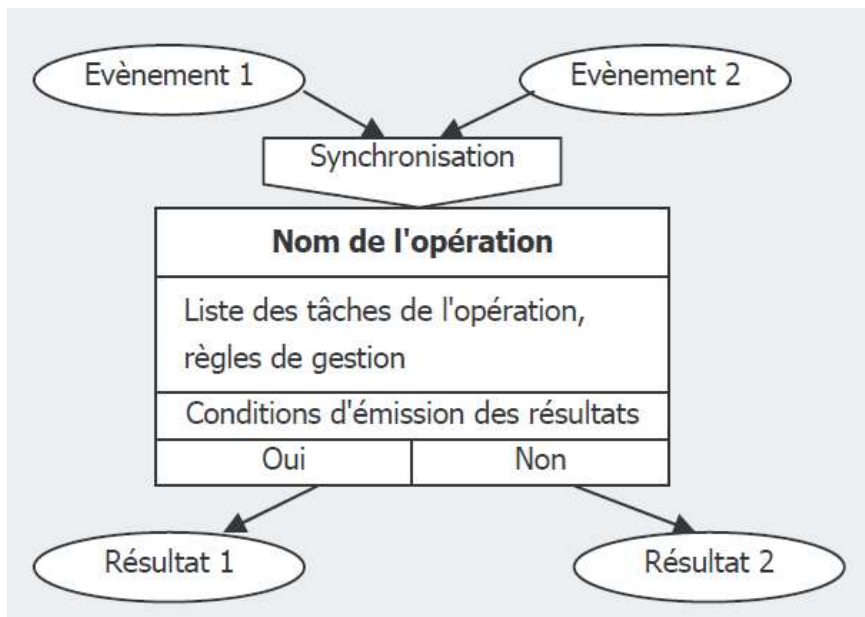
Ce diagramme montre l'ensemble des flux qui sont ensuite détaillés par le MCT



EXEMPLE

Le modèle conceptuel des traitements

Dans le modèle Merise, le M.C.T. est appelé **Modèle Évènement-Résultat** : L'arrivée d'un ou plusieurs événements va déclencher une opération qui va produire un résultat.



NIVEAU LOGIQUE OU D'ORGANISATION : À CE NIVEAU DE PRÉOCCUPATION, LES MODÈLES CONCEPTUELS SONT PRÉCISÉS ET FONT L'OBJET DE CHOIX D'ORGANISATION.

Le modèle logique des données ^{MLD}

- reprend le contenu du MCD précédent, mais précise la volumétrie, la structure et l'organisation des données telles qu'elles pourront être implémentées.
- Par exemple, connaissance de la liste exhaustive des tables à créer dans une Base de données.

Le modèle logique des traitements ^{MLT}

- précise les acteurs et les moyens qui seront mis en œuvre. C'est ici que les traitements sont découpés en procédures fonctionnelles (ou PF)..

NIVEAU PHYSIQUE : LES RÉPONSES APPORTÉES À CE DERNIER NIVEAU PERMETTENT D'ÉTABLIR LA MANIÈRE CONCRÈTE DONT LE SYSTÈME SERA MIS EN PLACE.

Le modèle physique des données ^{MPD}

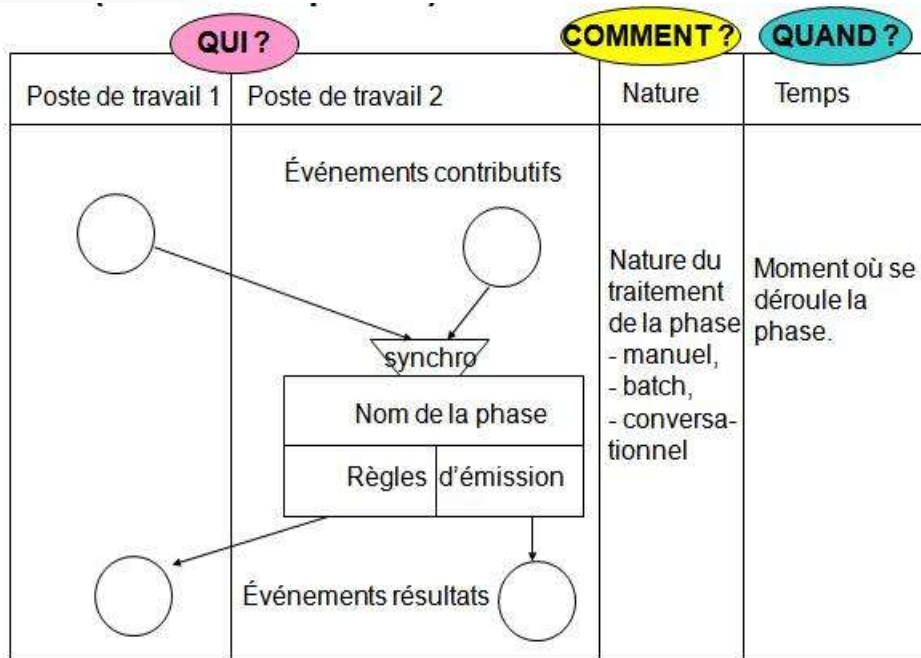
- permet de préciser les systèmes de stockage employés (implémentation du MLD dans le système de gestion de base de données retenu).

Le modèle opérationnel des traitements ^{MOT}

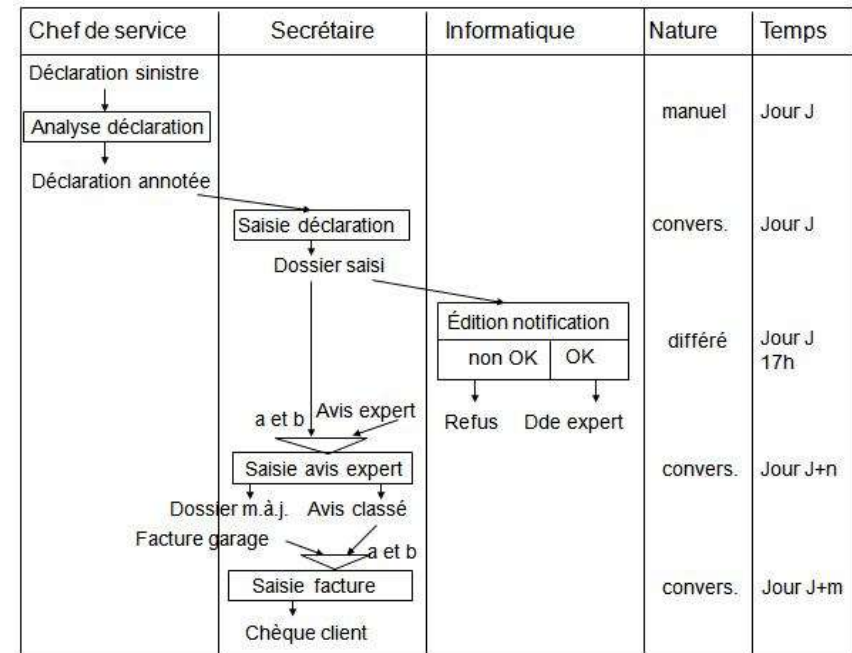
- permet de spécifier les fonctions telles qu'elles seront ensuite réalisées par le développeur.

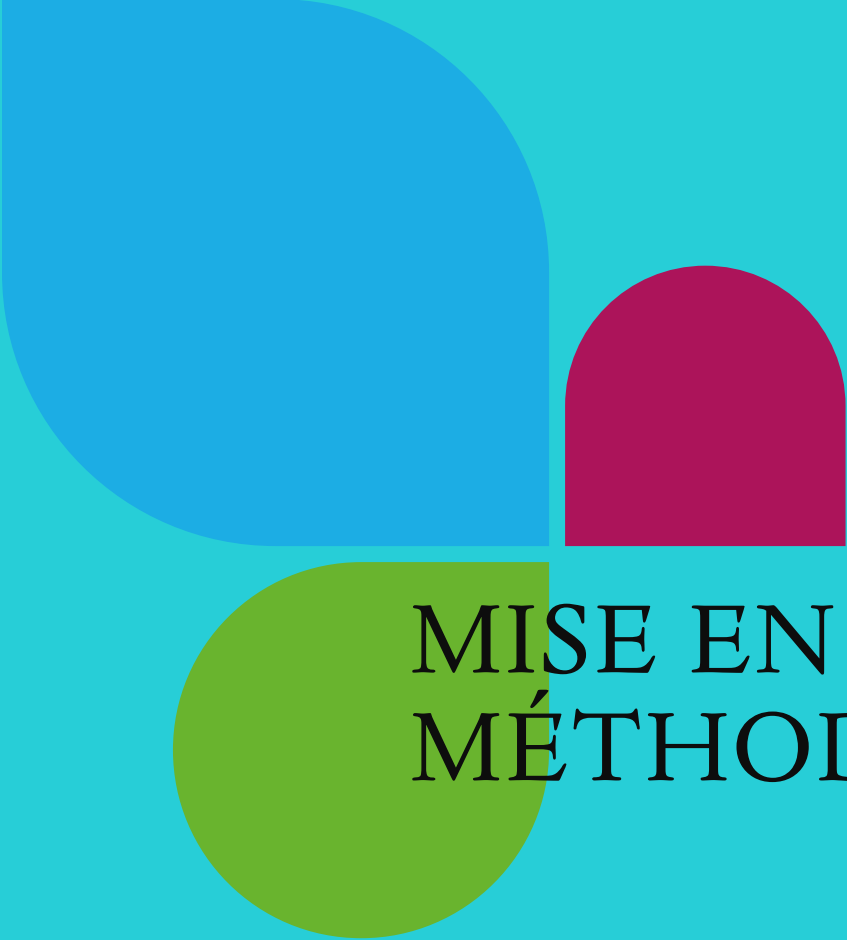
EXEMPLE

Le modèle opérationnel des traitements



Représentation graphique d'un MOT, avec un exemple.





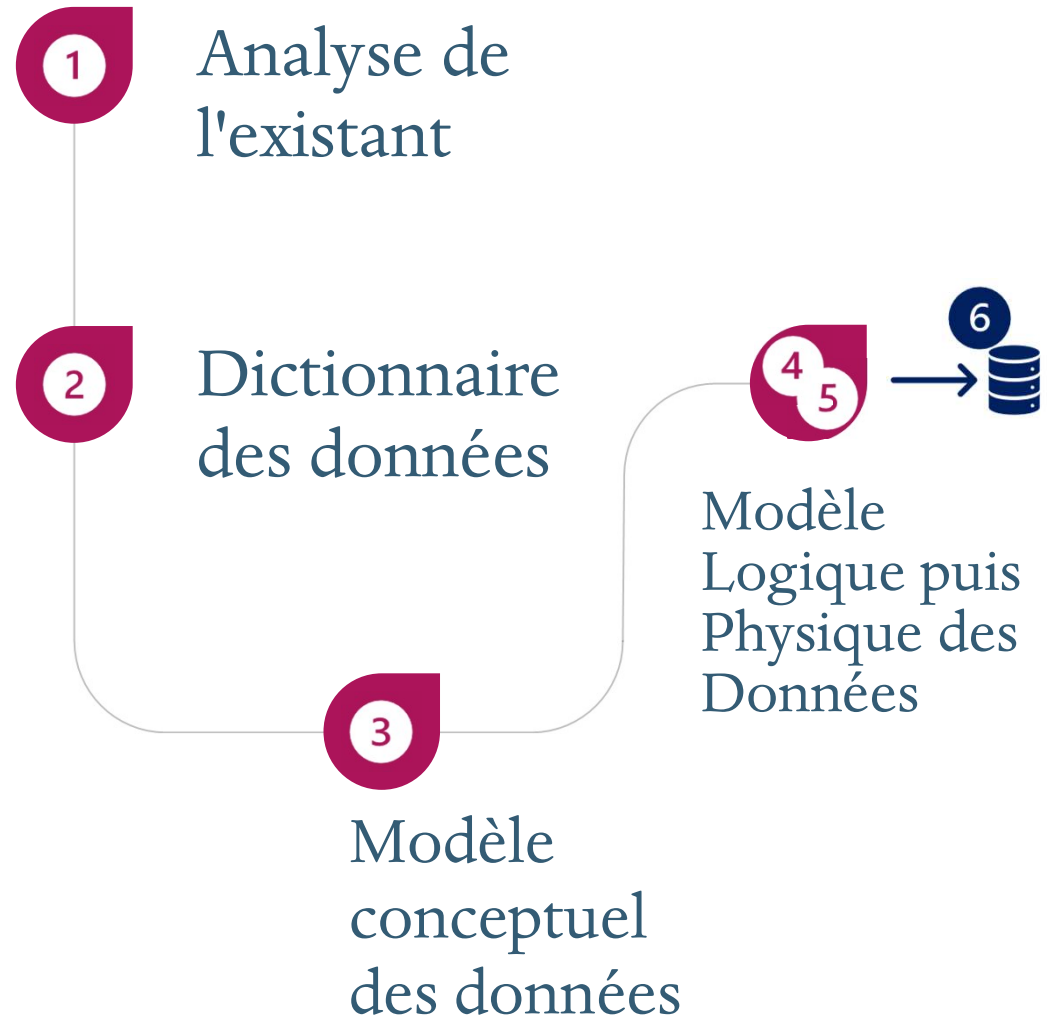
MISE EN OEUVRE DE LA MÉTHODE MERISE

ÉTAPE D'ÉLABORATION

Conception de notre future base de données.

Pour mettre en œuvre Merise et concevoir notre base de données de notre SI, il faut passer par quelques étapes d'élaboration.

Étape 6 : pour cette dernière étape, c'est écrire le script SQL mais pour cela, il faudra attendre de découvrir SQL





ETAPE 1 - ANALYSE DE L'EXISTANT

ANALYSE DE L'EXISTANT

La première étape, dans la construction d'une base de données, consiste à réunir tous les documents et les divers autres supports représentatifs des données que l'on souhaite modéliser.

Les bases de données existantes sont des points de départ intéressants. Mais cette source peut être complétée :

- Par l'analyse des documents papiers utilisés dans le système étudié,
- Par l'interview des utilisateurs.



- Avant de vous lancer dans la création de vos tables (ou même de vos entités et associations pour rester dans un vocabulaire conceptuel), il vous faut recueillir les besoins des futurs utilisateurs de votre application.
 - Utilisation de la méthode de conception Domain-Driven Design par exemple
- Et à partir de ces besoins, vous devez être en mesure d'établir des règles sur les données identifiées pendant l'analyse.
- Il existe deux types de règles :
 - Règles de gestion.
 - Règles d'organisation.

CLASSIFICATION DES RÈGLES

Règles de gestion Contraintes métiers (quoi)

Définition

- Ce sont des règles qui **définissent les contraintes métiers** d'une organisation. Elles expriment **ce qui est autorisé ou interdit** dans le cadre des activités de l'entreprise, en lien avec ses objectifs stratégiques, ses obligations légales, ou ses politiques internes.

Objectif

- Garantir que les processus métiers respectent les **exigences fonctionnelles** (ex : une commande ne peut être validée que si le stock est suffisant).

Exemples :

- Un client ne peut pas passer une commande si son solde est négatif.
- Une facture doit être payée sous 30 jours.
- Un produit ne peut être expédié que s'il est en stock.

Règles d'organisation Contraintes opérationnelles (qui, comment, temps)

Définition

- Ce sont des règles qui **définissent comment les activités sont organisées** dans l'entreprise. Elles précisent **qui fait quoi, quand, et avec quels moyens**, en structurant les rôles, les responsabilités, et les procédures.

Objectif

- Optimiser l'efficacité des processus en clarifiant les **responsabilités**, les **circuits de validation**, et les **modalités d'exécution**.

Exemples :

- Le service commercial valide les commandes avant envoi au service logistique.
- Le responsable RH doit signer les contrats de travail avant leur envoi.
- Les inventaires de stock sont réalisés tous les trimestres par le magasinier.

COMMENT LES IDENTIFIER ?

Pour les règles de gestion

- Posez des questions comme :
 - Quelles sont les conditions pour qu'une action soit valide ?
 - Quelles sont les obligations légales ou stratégiques ?
- Utilisez les **MCT** (Modèles Conceptuels de Traitement) pour formaliser les règles de gestion

Pour les règles d'organisation

- Posez des questions comme :
 - Qui est responsable de cette tâche ?
 - Quel est le circuit de validation ?
 - Quels sont les délais ou les outils utilisés ?
- Utilisez les **MOT** (Modèles Organisationnels de Traitement) pour les règles d'organisation.

EXERCICE

Marquez les règles suivantes d'un G ou d'un O selon que vous avez affaire à une règle de Gestion ou d'Organisation :

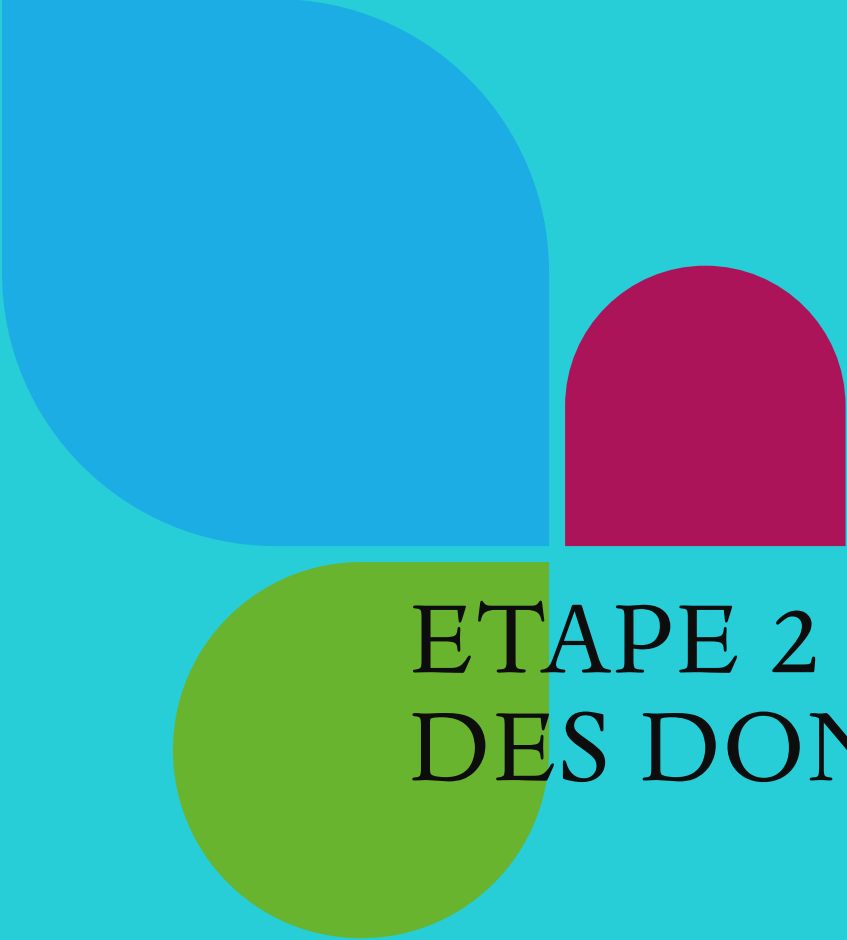
Réponses

1. O
2. O
3. G
4. O
5. G
6. G
7. O
8. O
9. G
10. G

Exercice

Règles de gestion ou Règles d'organisation ?

1. Le service achats et les magasins sont équipés d'ordinateurs pouvant s'échanger des données. Le service commercial dispose d'un matériel analogue.
2. La M.A.J. du stock s'effectue :
 - le matin à 9 hrs pour les sorties de stock
 - en temps réel à tout autre moment de la journée
3. Un produit peut être en stock dans plusieurs magasins.
4. Le courrier est expédié à 12 hrs.
5. Une commande de réapprovisionnement concerne un fournisseur.
6. On tient à jour un stock théorique d'après les mouvements du stock.
7. Dans un magasin, tout produit doit pouvoir être rangé dans un seul casier et tout casier ne doit contenir qu'un seul produit.
8. A chaque livraison fournisseur, le magasinier contrôle la marchandise livrée en la comparant à la marchandise commandée figurant sur la commande au fournisseur.
9. Les livraisons des fournisseurs sont contrôlées par comparaison avec les commandes. Toute livraison non conforme est refusée et retournera chez le fournisseur.
10. Les mouvements de stock sont :
 - Hors période d'inventaire.
 - Livraison Fournisseur : $\text{Stock} = \text{Stock} + \text{Qté livrée}$
 - Bon de livraison Client : $\text{Stock} = \text{Stock} - \text{Qté livrée}$
 - Retour marchandises Client : $\text{Stock} = \text{Stock} + \text{Qté retournée}$
 - Pendant ou hors période d'inventaire :
 - Ajustement (suite à inventaire ou à écart occasionnel constaté)
 - $\text{Stock} = \text{Stock} + / - \text{écart entre stocks réel et théorique}$. Les retours marchandises fournisseurs n'entrent pas en jeu car les marchandises sont retournées avant d'avoir été prises en compte dans le stock théorique.



ETAPE 2 - DICTIONNAIRE DES DONNÉES.

DICTIONNAIRE DE DONNÉES.

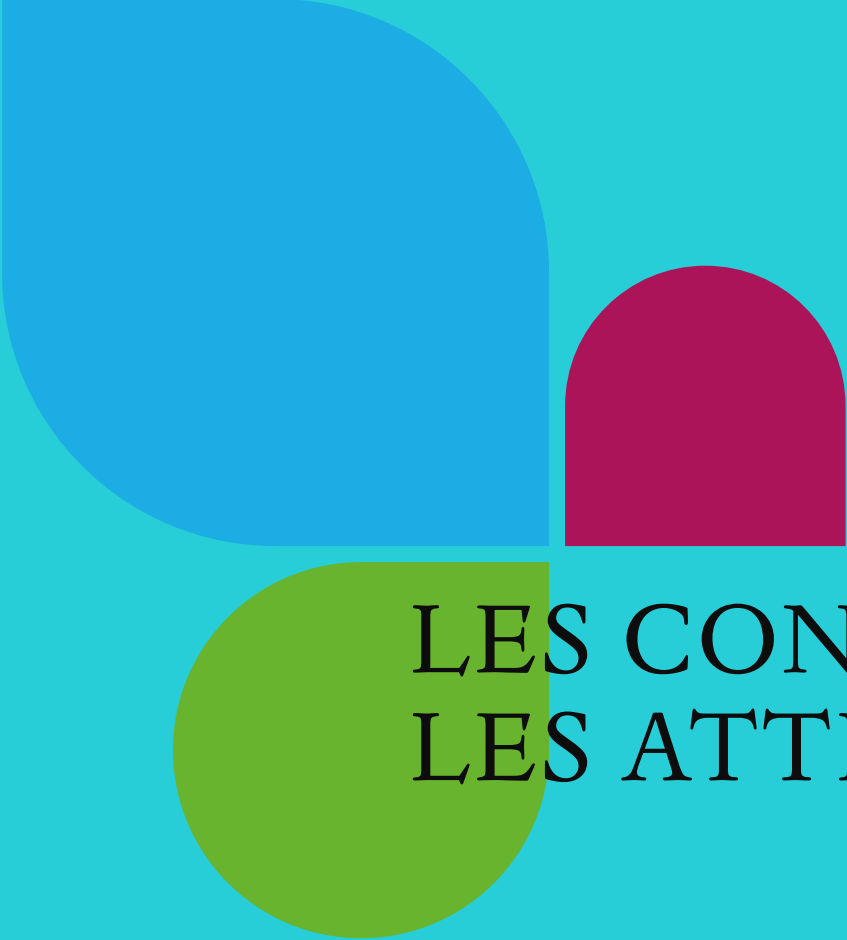
Partant des documents, il permet la phase d'analyse à en extraire les informations élémentaires (entités, attributs, type, règles) qui vont constituer la future base de données.

Les données figurant dans le dictionnaire doivent être, dans la plupart des cas, élémentaires :

- **elles ne doivent pas être calculées** : données calculées à partir de données élémentaires.
- **elles ne doivent pas être composées** : données composées par la concaténation de données élémentaires.

Si on n'effectue pas de calcul sur une donnée numérique, celle-ci doit être de type Alphanumérique. (téléphone par exemple).

Propriété	Signification
Code	Abréviation du nom de l'attribut en respectant les normes de nommage. La première lettre du nom est en minuscule. La première lettre de chaque mot est en majuscule, les autres en minuscule. Choisir des noms explicites qui décrivent facilement la donnée. Il n'est pas utile de se limiter dans la taille des noms si ce n'est que ce sont ces codes qui sont utilisés pour le codage des composants.
Libellé	Libellé contenant la signification précise et le rôle de l'attribut. Ce doit être un nom communément utilisé pour désigner l'objet qui ne risque pas d'introduire un contresens.
Type de donnée	Type de l'attribut : entier, réel, chaîne de caractères, date...
Contraintes d'intégrité	Liste des contraintes sur les valeurs possibles de l'attribut
Règle de calcul	Règle de calcul (d'obtention) de l'attribut correspondant



LES CONTRAINTES SUR LES ATTRIBUTS

CONTRAINTES SUR LES VALEURS DES ATTRIBUTS

Contrainte de domaine

Elle définit le domaine ou l'ensemble de valeurs d'un attribut et garantit que la valeur prise par l'attribut doit être une valeur atomique (ne peut pas être divisée) de son domaine.

- not null
- check (ex : salaire > 0)
- Unique, default
- primary key, foreign key
- index

Contrainte de borne et de liste

Elle définit une borne [x à y] ou une liste de valeurs [a, b, c] qu'un attribut peut prendre, garantissant la conformité des données qui seront ensuite enregistrées dans la future base de données.

EXEMPLE DE DICTIONNAIRE DES DONNÉES

Code mnémorique	Désignation	Type	Taille	Remarque
number	Matricule du collaborateur	Numérique	4	Incrémentation séquentielle - unique - non modifiable
civilRegistration	Etat civil du collaborateur	varchar	3	MR : Monsieur MME : Madame MLE : Mademoiselle
sex	Sexe du collaborateur	char	1	F : féminin M : Masculin
lastName	Nom du collaborateur	varchar	20	
firstName	Prénom du collaborateur	varchar	20	
addr1	Adresse ligne 1 collaborateur	varchar	25	
addr2	Adresse ligne 2 complémentaire	varchar	25	
city	Ville du collaborateur	varchar	20	
postCode	Code postal du collaborateur	varchar	5	
phoneEmployee	Téléphone du collaborateur	varchar	10	
hobbies	Hobbies du collaborateur	varchar	200	
idContract	Identifiant du contrat du collaborateur	Numérique	4	Incrémentation séquentielle - unique - non modifiable
status	Statut du collaborateur	varchar	3	
salary	Salaire du collaborateur	décimal		
entryDate	Date d'embauche du collaborateur	date		
exitDate	Date de fin de contrat du collaborateur	date		
idFunction	Identifiant de la fonction	Numérique	2	Incrémentation séquentielle - unique - non modifiable
labelFunction	Fonction du collaborateur	varchar	15	(Responsable de projet - Analyste responsable d'application - Développeur - Technicien support - Commercial - Technico-commercial - Secrétaire)

IDENTIFIER LES ENTITÉS

Avec l'analyse du cahier des charges, cela permet d'extraire les futures données de la BDD et notamment les identifiants permettant d'offrir l'unicité des données dans la future base de données.

Ces identifiants permettront d'imaginer les futures entités que nous pourrions construire dans l'étape suivant qu'est le MCD.

A partir d'un identifiant, on doit pouvoir connaître les dépendances fonctionnelles : c'est-à-dire quelles données obtenues à partir d'un identifiant.

De cette nouvelle analyse, j'obtiens mes entités.

identifiant

number	Matricule du collaborateur	Numérique	4	Incrémentation séquentielle - unique - non modifiable
civilRegistration	Etat civil du collaborateur	varchar	3	MR : Monsieur MME : Madame MLE : Mademoiselle
sex	Sexe du collaborateur	char	1	F : féminin M : Masculin
lastName	Nom du collaborateur	varchar	20	
firstName	Prénom du collaborateur	varchar	20	
addr1	Adresse ligne 1 collaborateur	varchar	25	
addr2	Adresse ligne 2 complémentaire	varchar	25	
city	Ville du collaborateur	varchar	20	
postCode	Code postal du collaborateur	varchar	5	
phoneEmployee	Téléphone du collaborateur	varchar	10	
hobbies	Hobbies du collaborateur	varchar	200	
idContract	Identifiant du contrat du collaborateur	Numérique	4	Incrémentation séquentielle - unique - non modifiable

Remarques : Il se peut que nous n'ayons pas d'attributs pouvant jouer le rôle d'identifiant, alors on a le droit de créer un nouvel attribut qui servira d'identifiant. Généralement, il sera de type entier.

EXEMPLE D'ANALYSE DES ENTITÉS

Par exemple : avec une valeur de idContract, je peux identifier les données des attributs status et salary. Pareil pour number qui me permettra d'identifier les données des attributs listés dans le tableau ci-dessous

DEPENDANCES FONCTIONNELLES DEDUITES ET ENTITES DEDUITES.

Clés	Attributs	Entités déduites
number	civilRegistration, sex, lastName, firstName, addr1, addr2, city, postCode, phoneEmployee, hobbies	Employee
idContract	status, salary	Contract



EXERCICES

GESTION DES STAGIAIRES

Comme exemple, la gestion de stagiaires qui suivent les différentes séances d'une formation. Les documents de travail utilisés sont les suivants :

Liste des stagiaires			
N° stagiaire	Nom de l'stagiaire	Date de naissance	Sexe
100	MARTIN, Paul	15/10/1975	H
200	DULAC, Georgia	25/12/1969	F
300	DUPONT, Michel	01/06/1985	H
...	...	...	...

Point d'avancement			
N° stagiaire : 100 Paul Martin			
N° séance	Libellé	Niveau d'apprentissage	Acquis/non acquis
1	Dictionnaire des données	Maîtrise	O
2	Modèle E/A	En-cours	N
3	Schéma de la base	initié	N
4	Langage SQL - LDD	initié	E
5	Langage SQL - LMD	Non démarré	N

Etablir le dictionnaire des données relatif aux documents ?

id formateur	Nom	Ancienneté	Séance enseignée
25ED001	Marc AUREL	2	Dictionnaire des données
29ED002	César JULIUS	3	Schéma de la base
...	...	...	...
55ED950	Parfait NERON	5	Langage SQL - LDD

III REGLES DE GESTION

- Une séance n'est enseignée que par un seul formateur.
- Un stagiaire n'a qu'un niveau d'apprentissage par séance (le dernier).

CORRECTION

CODE	LIBELLE	TYPE	TAILLE	Contrainte d'intégrité	REGLE O
numeroStagiaire	numero du stagiaire	entier		unique - non nulle	
nomStagiaire	nom du stagiaire	chaîne de caractère	25	non nulle	
prenomStagiaire	prenom du stagiaire	chaîne de caractère	25	non nulle	
dateNaissance	date de naissance du stagiaire	date		non nulle	verifier > 18
sexe	sexe du stagiaire	caractere	1	non nulle	H - F
idFormateur	identifiant du formateur	chaîne de caractère	7	non nulle - unique	
nomFormateur	nom du formateur	chaîne de caractère	25	non nulle	
prenomFormateur	prenom du formateur	chaîne de caractère	25	non nulle	
anciennete	ancienneté du formateur	entier			
nomSeance	libellé de la séance	chaîne de caractère	50	non nulle	une seance n'est enseigné par un seul formateur
numeroSeance	identifiant de la seance	entier		non nulle - unique	
idEvaluation	identifiant de l'evaluation de la seance pour un stagiaire	entier		non nulle - unique	
niveauApprentissageStagiaire	Niveau d'apprentissage du stagiaire	Enumeration		Maitrise - En-cours - initié - non démarré	
Acquisition	Acquis/Non Acquis	Enumeration		O - N - E	
					ENTITES DEDUITES
numeroStagiaire	nomStagiaire	prenomStagiaire	dateNaissance	sexe	Stagiaire
idFormateur	nomFormateur	prenomFormateur	anciennete		Formateur
numeroSeance	nomSeance	niveauApprentissageStagiaire	Acquisition		Seance
idEvaluation	niveauApprentissage	Acquisition			Evaluation

EXERCICES METIS

Réaliser les exercices METIS suivants :

- 1.CAS "SUIVI DES PLAGES"
- 2.CAS "GRAND PRIX"
- 3.CAS "RESTAURANT"

CORRECTION CAS PLAGE

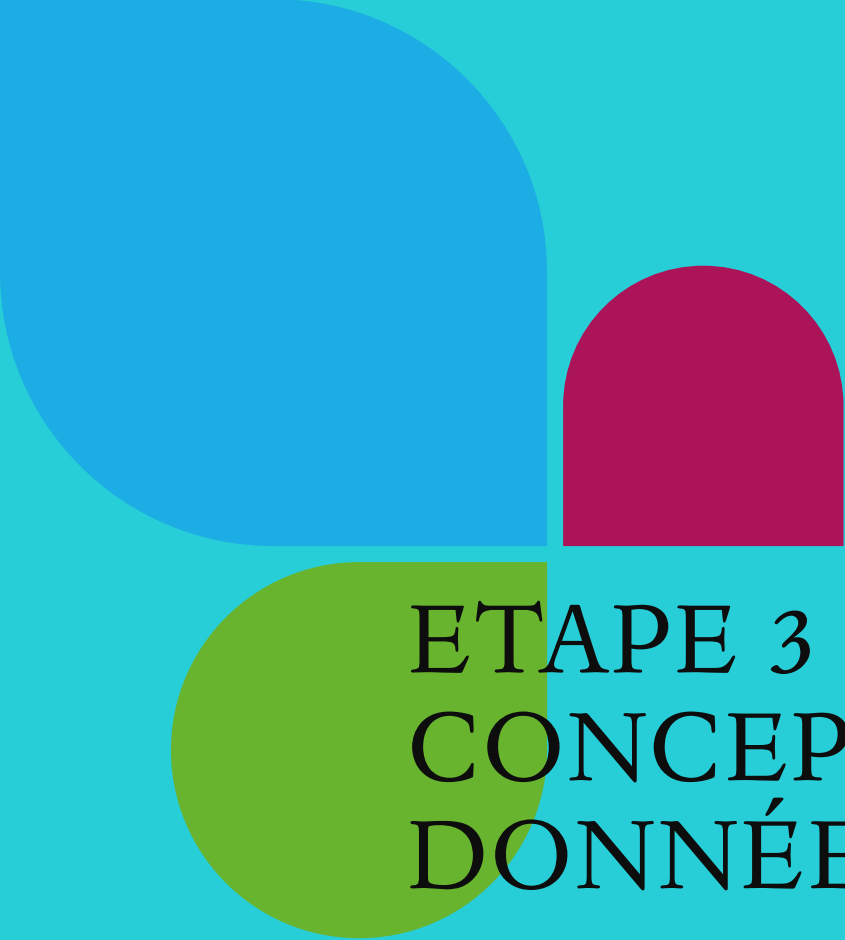
1ère étape	Identifier les attributs			
CODE	Description	Type	taille	Contrainte
nomRegion	nom de la Région	String	50	not null
nomResponsable	nom du Responsable	String	50	not null
prenomResponsable	prenom du Responsable	String	50	not null
nomPlage	nom de la plage	String	50	not null
nombreTouristeAnnuel	nombre de touriste annuel reçu	entier		>=0
LongueurPlage	longueur de la plage	entier		>0
nomVille	nom de la ville	String	50	not null
codePostal	code postal de la ville	entier	5	not null
nomDepartement	nom du Département	String	50	not null
nomTypeTerrain	libellé du type de terrain	String	50	not null

CORRECTION CAS PLAGE

2eme étape	Repérer les possibles identifiants			
CODE	Description	Type	taille	Contrainte
idRegion	identifiant de la région	entier		autoincrément, >0, not null
nomRegion	nom de la Région	String		not null
idResponsable	identifiant du responsable	entier		autoincrément, >0, not null
nomResponsable	nom du Responsable	String		not null
prenomResponsable	prenom du Responsable	String		not null
idPlage	identifiant de la place	entier		autoincrément, >0, not null
nomPlage	nom de la plage	String		not null
nombreTouristeAnnuel	nombre de touriste annuel reçu	entier		>=0
LongueurPlage	longueur de la plage	entier		>0
IdVille	identifiant de la ville	entier		autoincrément, >0, not null
nomVille	nom de la ville	String		not null
codePostal	code postal de la ville	entier	5	not null
idDepartement	identifiant du département	entier		autoincrément, >0, not null
nomDepartement	nom du Département	String		not null
idTypeTerrain	identifiant du type de terrain	entier		autoincrément, >0, not null
nomTypeTerrain	libellé du type de terrain	String		not null

CORRECTION CAS PLAGE

3eme étape	vérifier l'unicité des données en contrôlant qu'avec un identifiant on cible les attributs voulus	
Identifiant	Attributs ciblés	Entité
idRegion	nomRegion, nomResponsable, prenomResponsable	Region
idPlage	nomPlage, nombreTouristeAnnuel, longueurPlage, natureTerrainPlage	Plage
idVille	nomVille, codePostal	Ville
idDepartement	nomDepartement	Departement
idTypeTerrain	typeTerrain	TypeTerrain
IdResponsable	nomResponsable, prenomResponsable	Responsable



ETAPE 3 - MODÈLE CONCEPTUEL DES DONNÉES.

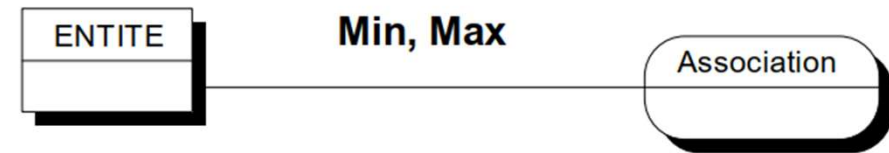
MODÈLE CONCEPTUEL DES DONNEES - MCD

À partir du dictionnaire des données, nous allons élaborer un modèle représentatif pour la structuration de données d'un système d'informations.

De ce modèle représentatif nommé **le Modèle Entité-Association**, on va avoir :

1. La représentation graphique de la structure de données.
2. Le repérage des **entités**,
3. Le repérage des **relations** entre entités : **les associations**
4. **L'identification** des entités (Unicité des entités),
5. Le positionnement des cardinalités Min,Max sur les entités.

MODÈLE ENTITÉ-ASSOCIATION



MODÈLE CONCEPTUEL DE DONNÉES

Entité

Une entité, représentant un objet abstrait, élémentaire et qui possède :

- Un nom **unique**
- Des attributs

A toute entité correspond une représentation graphique sous forme de table avec

- Un nom
- Des attributs

PERSONNE
<u>id_personne</u>
nom_personne
prenom_personne

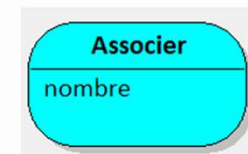
Association

Une association décrit une relation existante entre plusieurs entités.

- Elle n'a de sens que sous réserve de l'existence des entités dont elle fait la relation.

Une association possède :

- un nom (souvent un verbe)
- et **éventuellement** des propriétés appelées **propriétés portées**.
- On dit de l'association alors qu'elle est porteuse de propriétés



IDENTIFICATION DES ENTITÉS

Clef identifiante

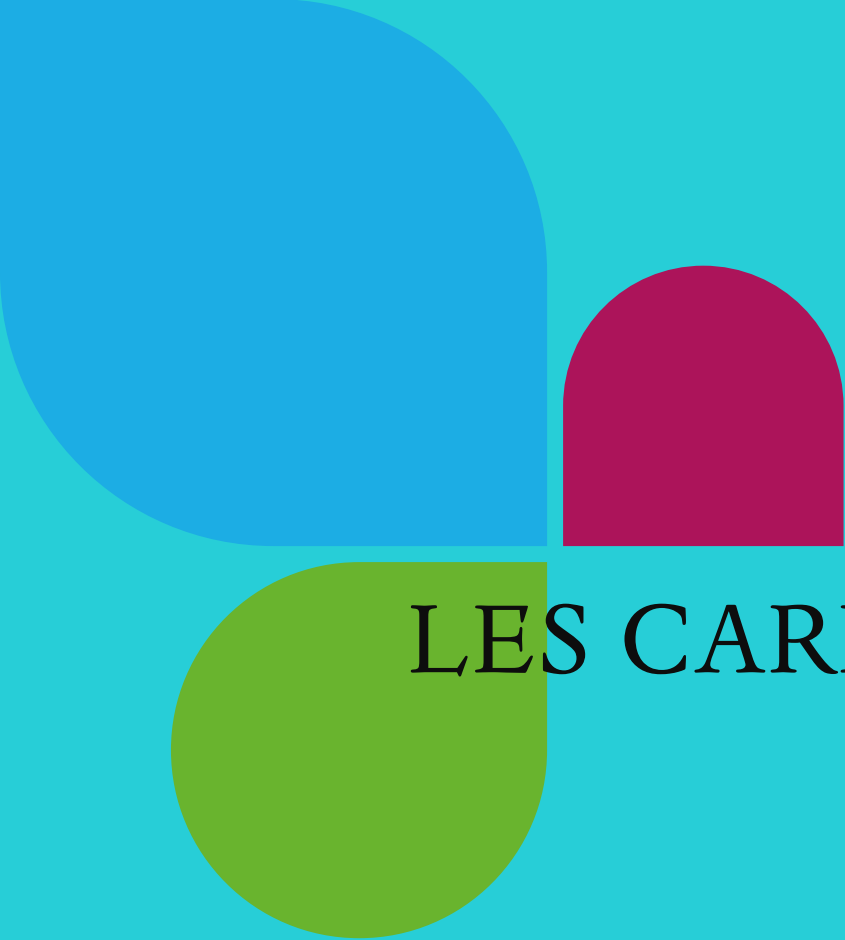
- la clef est l'identifiant de la ligne.
 - **Toute entité possède une clef.**
- Lorsque la clef est constituée de plusieurs propriétés, on parle *de clef composée*.

! Important !

- La clef d'une entité doit figurer dans les propriétés de l'entité. Elle est soulignée.
- **Dans des cas bien précis**, la clef correspondant à une association est sous-entendue.
 - Elle est constituée de la clef identifiante de chacune des entités qu'elle met en relation.

Contraintes sur la clé identifiante

- **unique** pour conserver l'unicité de l'entité. Il est propre à chaque ligne de la table.
- **non null** : en effet, la clé primaire doit toujours correspondre à une valeur.
- son type est généralement un **entier** mais il est possible d'utiliser d'autres types
- elle est **auto-incrémentée**.



LES CARDINALITÉS

LES CARDINALITÉS

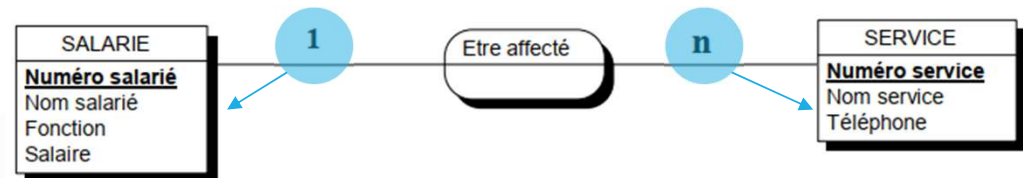
- Couples de valeur entre chaque entité et ses associations liées.
- Pour une association de 2 entités soit 4 cardinalités à indiquer (2 de chaque côté).
 - **Min , Max**
- Trois valeurs typiques : **0, 1, n (plusieurs)**.
 - Permettent d'indiquer combien de fois au minimum et au maximum une occurrence d'entité peut être liée à une autre occurrence d'entité.
- Les cardinalités traduisent des règles de gestion. Ces règles expriment des contraintes sur le modèle.

Exemple

RG (règles de gestion)

Un salarié est affecté au plus à un seul service.

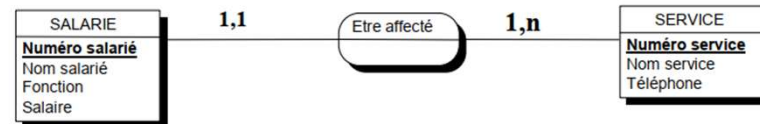
Dans un service sont affectés plusieurs salariés



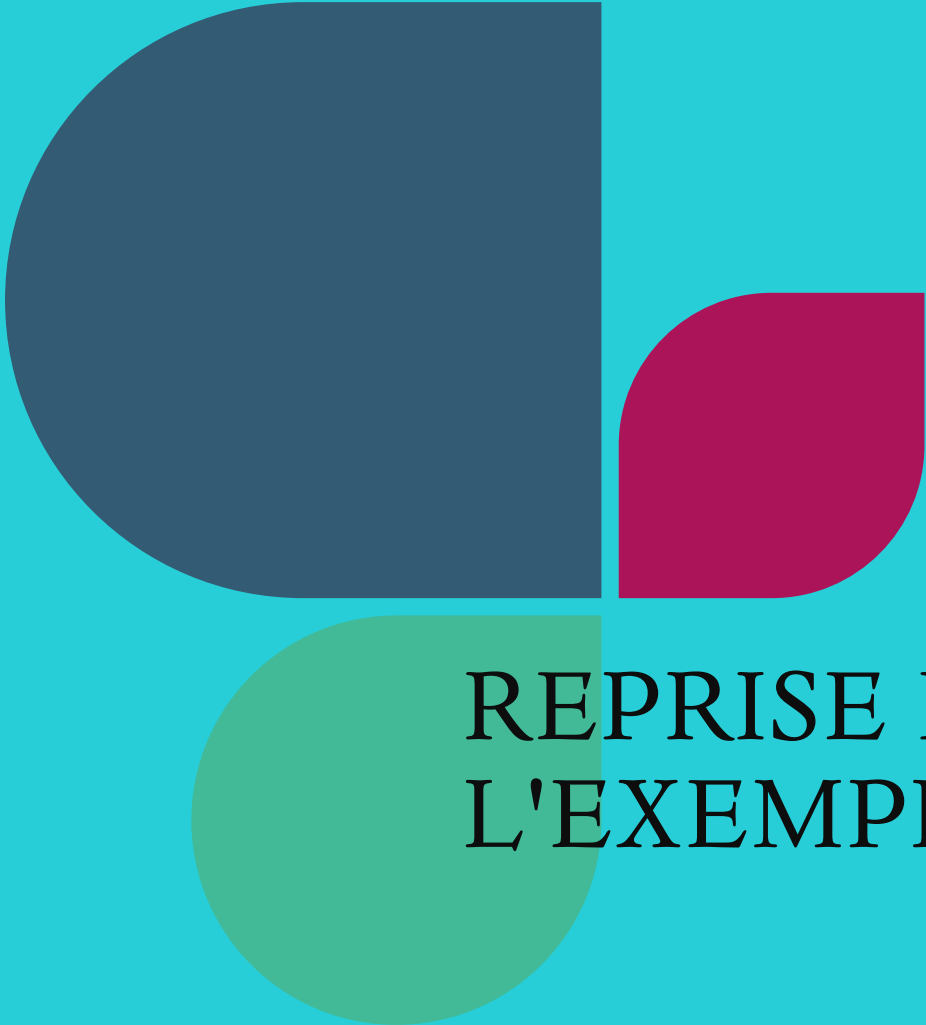
RÉCAPITULATIF SUR LES CARDINALITÉS

En fait, on n'utilise que 4 combinaisons de valeurs pour les cardinalités :

- 0,1 : au plus un(e)
- 1,1 : un(e) et un(e) seul(e)
- 1,n : un(e) ou plusieurs
- 0, n : zéro ou plusieurs



RG : Un salarié est obligatoirement affecté à 1 et 1 seul service. Un service pour exister doit avoir au moins un salarié affecté (sous-entendu, il peut bien sur en avoir plusieurs).



REPRISE DE L'EXEMPLE STAGIAIRE

DICTIONNAIRE DES DONNÉES

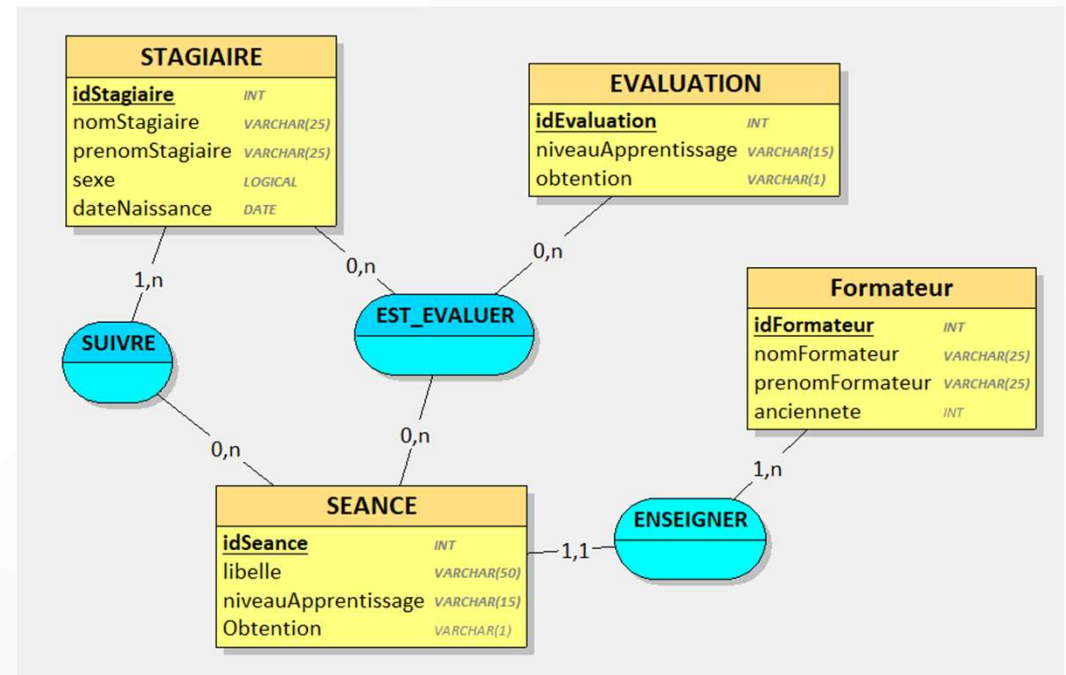
CODE	LIBELLE	TYPE	TAILLE	Contrainte d'intégrité	REGLE O
numeroStagiaire	numero du stagiaire	entier		unique - non nulle	
nomStagiaire	nom du stagiaire	chaîne de caractère	25	non nulle	
prenomStagiaire	prenom du stagiaire	chaîne de caractère	25	non nulle	
dateNaissance	date de naissance du stagiaire	date		non nulle	verifier > 18
sexe	sexe du stagiaire	caractere	1	non nulle	H - F
idFormateur	identifiant du formateur	chaîne de caractère	7	non nulle - unique	
nomFormateur	nom du formateur	chaîne de caractère	25	non nulle	
prenomFormateur	prenom du formateur	chaîne de caractère	25	non nulle	
anciennete	ancienneté du formateur	entier			
nomSeance	libellé de la séance	chaîne de caractère	50	non nulle	une seance n'est enseigné par un seul formateur
numeroSeance	identifiant de la seance	entier		non nulle - unique	
idEvaluation	identifiant de l'evaluation de la seance pour un stagiaire	entier		non nulle - unique	
niveauApprentissageStagiaire	Niveau d'apprentissage du stagiaire	Enumeration		Maitrise - En-cours - initié - non démarré	
Acquisition	Acquis/Non Acquis	Enumeration		O - N - E	
					ENTITES DEDUITES
numeroStagiaire	nomStagiaire	prenomStagiaire	dateNaissance	sexe	Stagiaire
idFormateur	nomFormateur	prenomFormateur	anciennete		Formateur
numeroSeance	nomSeance	niveauApprentissageStagiaire	Acquisition		Seance
idEvaluation	niveauApprentissage	Acquisition			Evaluation

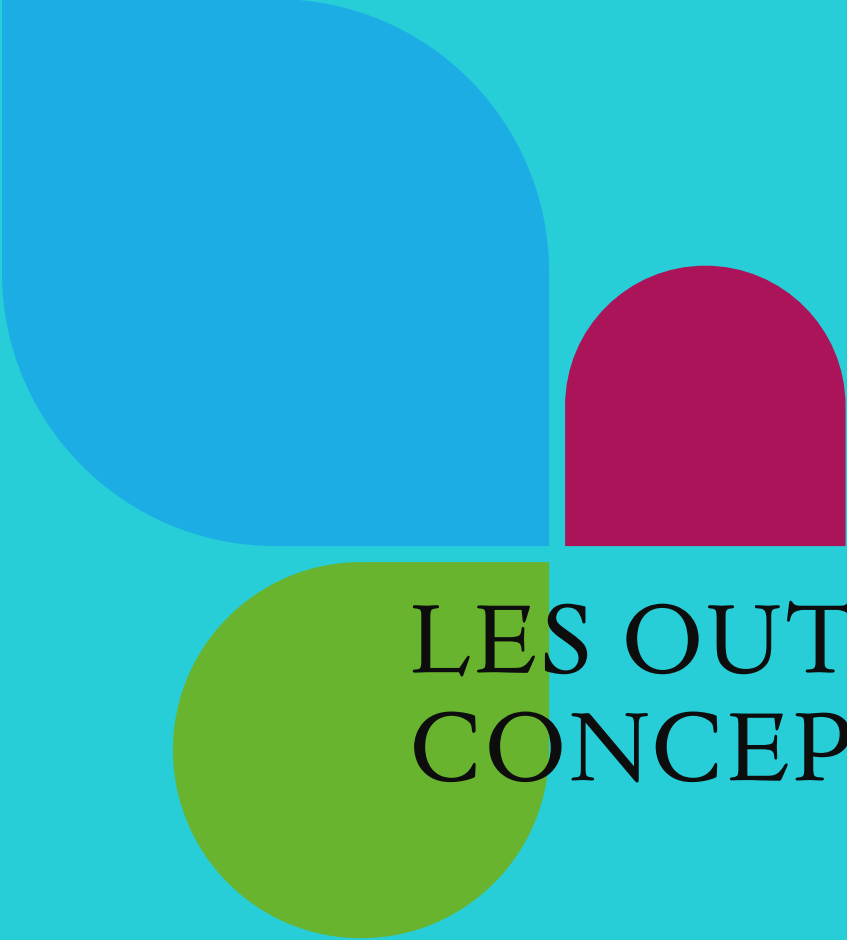
IMAGINONS LE MCD

En reprenant notre analyse et notre dictionnaire de données, je peux mettre en place le **Modèle Entité Association** suivant :

PS : schéma réalisé avec Looping

<https://www.looping-mcd.fr/>



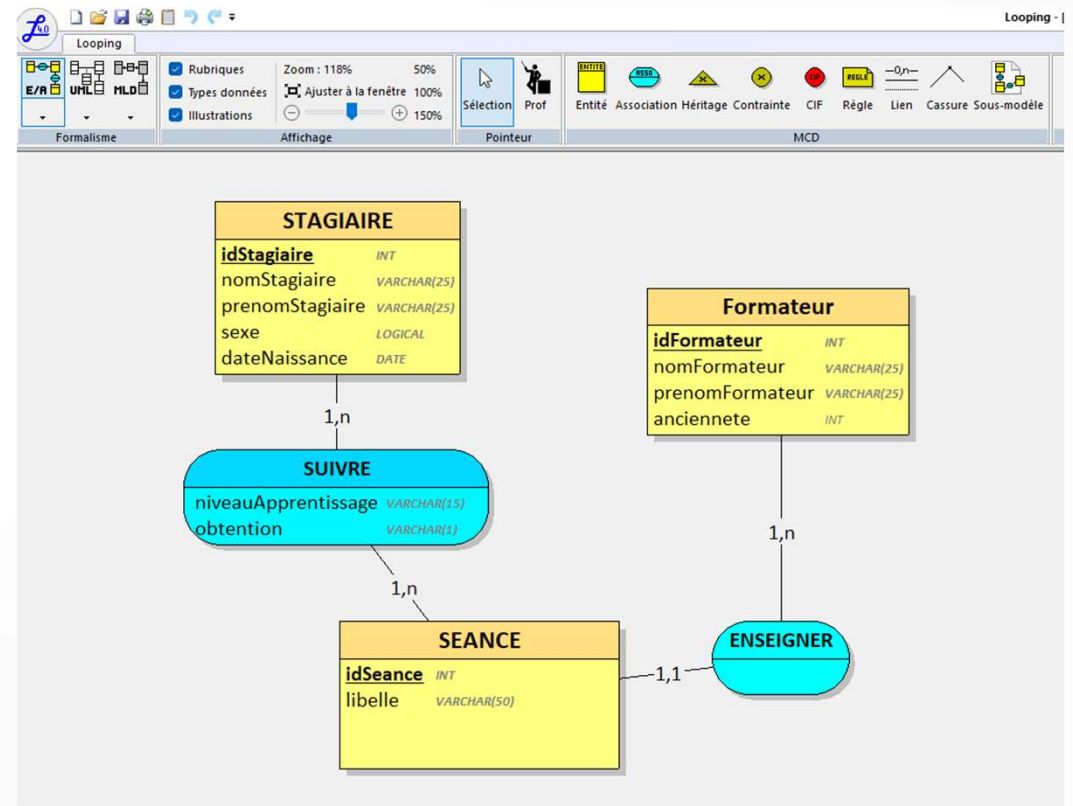


LES OUTILS DE CONCEPTION

LOOPING

<https://www.looping-mcd.fr/>

Le plus : gratuit

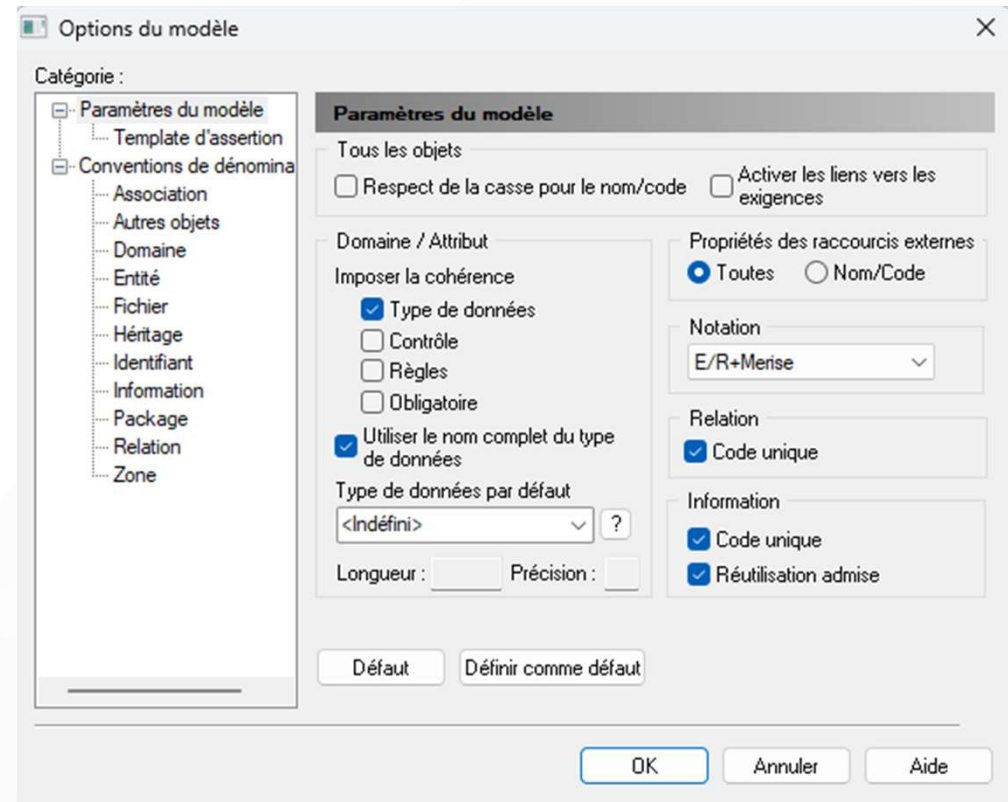


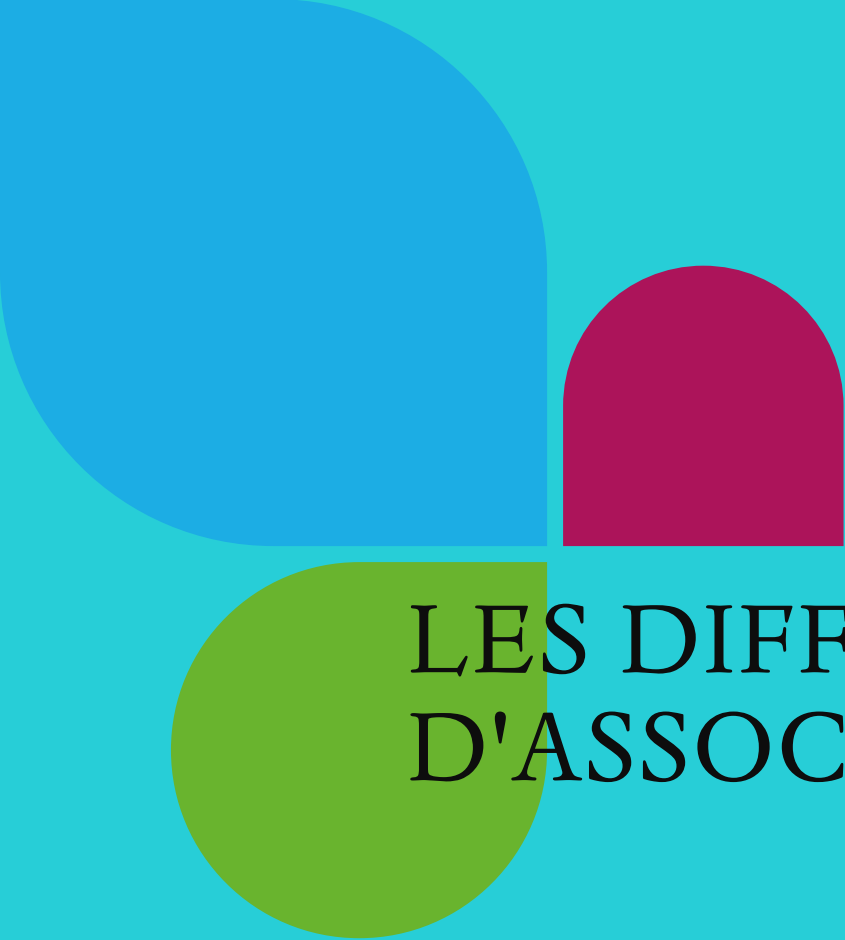
POWER DESIGNER

Le plus : professionnel

Le moins : payant

1. Outils
 1. Options du modèle
 2. Notation = E/R+Merise
2. Créer un projet
3. Clic droit sur le projet
 1. Ajouter un nouveau modèle
 2. Modèle conceptuel des données





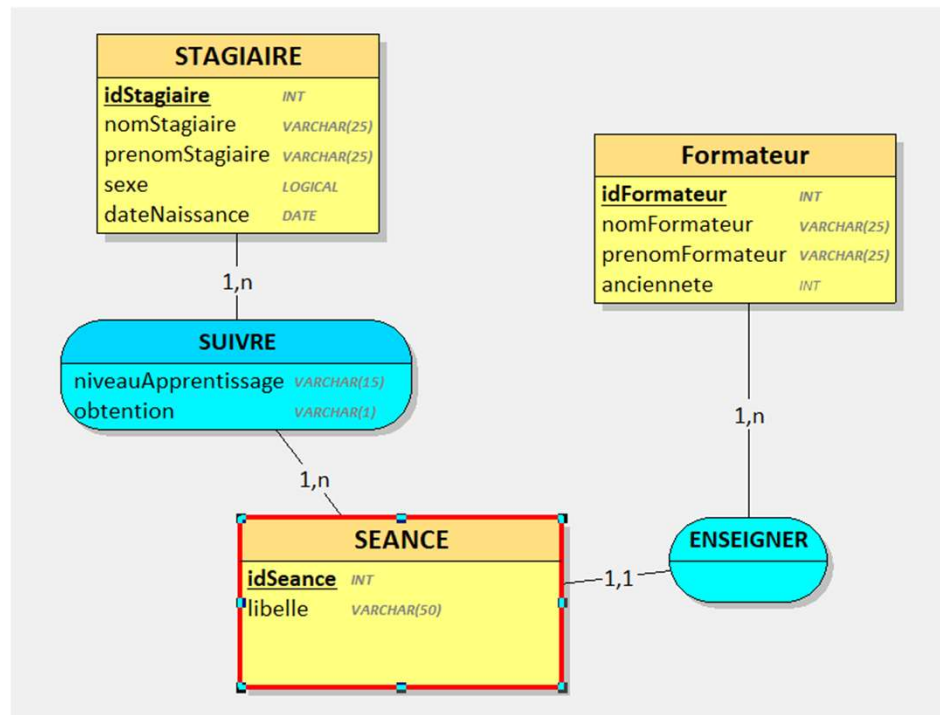
LES DIFFÉRENTES TYPES D'ASSOCIATIONS

TYPES D'ASSOCIATIONS

En Merise, les associations représentent les liens entre les **entités** (objets du monde réel modélisés dans le système d'information).

- **Association Binaire**
 - **Définition** : Une association entre deux entités uniquement.
 - **Exemple** : Un client passe une commande.2.
- **Association Ternaire (ou n-aire)**
 - **Définition** : Une association entre trois entités ou plus.
 - **Exemple** : Un enseignant donne un cours dans une salle.3.
- **Association Réflexive**
 - **Définition** : Une association où une entité est en relation avec elle-même.
 - **Exemple** : Un employé est le manager d'un autre employé.
- **Association avec Attributs (voir exemple)**
 - **Définition** : Une association qui possède ses propres attributs (informations supplémentaires).
 - **Exemple** : Une réservation (association entre un client et une chambre) a une date de début et une date de fin.
- **Association de Spécialisation/Généralisation**
 - **Définition** : Une relation de type "est-un" entre une entité générale et une entité spécialisée.
 - **Exemple** : Un véhicule (entité générale) peut être une voiture ou un camion (entités spécialisées).

ASSOCIATIONS PORTEUSES D'ATTRIBUTS.



En reprenant notre 1er MCD, je peux faire porter les attributs **niveauApprentissage** et **obtention** sur l'association SUIVRE, on dit alors que notre association est porteuse d'attribut

Attention : Cela n'est possible qu'avec une cardinalité x-n entourant l'association sinon les attributs seront perdues !!

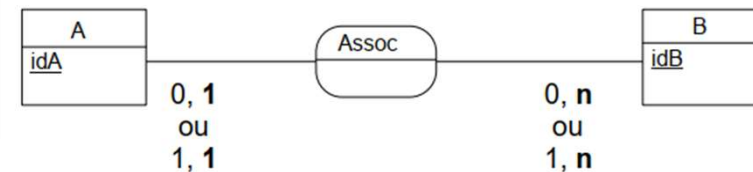
ASSOCIATIONS HIÉRARCHIQUES [1,N]

Les associations hiérarchiques se caractérisent par une relation où une entité **parent** peut avoir plusieurs entités **enfant**, mais chaque enfant n'a qu'un seul parent.

Exemple d'association hiérarchique :

- Un employé peut avoir plusieurs subordonnés, mais chaque subordonné a un seul manager.

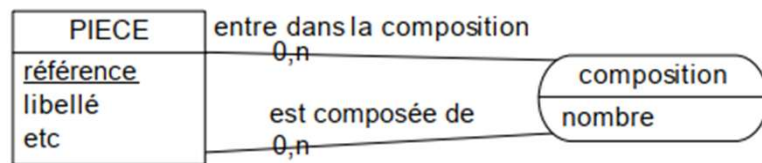
- Association ayant d'un côté, la cardinalité max à 1, et de l'autre côté, la cardinalité max à n.
- Cela se traduit par exemple :
 - un fils (A) n'a qu'un seul père (B) et un père (B) peut avoir plusieurs fils (A)
 - C'est un lien de dépendance fonctionnelle (DF). Un fils détermine bien le père.
 - Il n'y a pas de propriétés à l'intérieur de l'association**



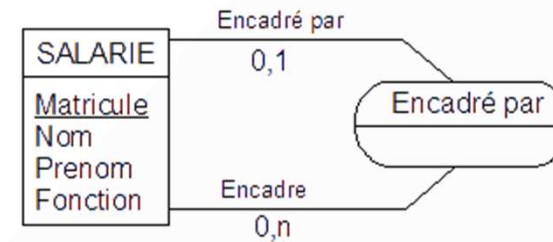
LES ASSOCIATIONS RÉFLEXIVES

Une association réflexive relie des occurrences d'une même entité.

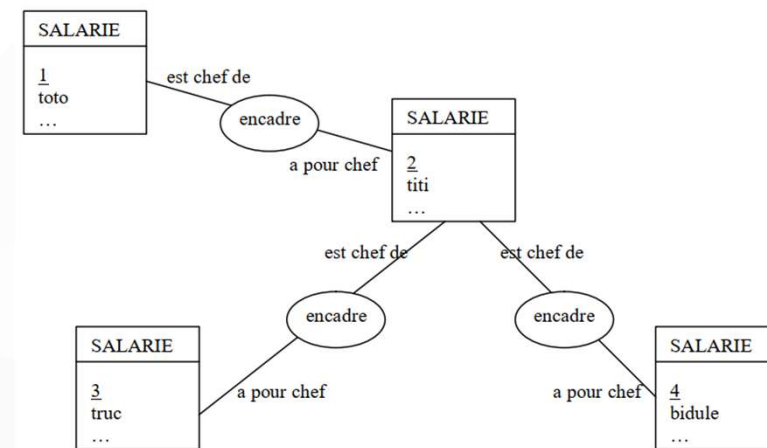
- Ces associations sont quasiment toujours binaire (2 branches).
- **réflexive hiérarchique**
 - par exemple, un employé qui supervise d'autres employés
- **réflexive non hiérarchique**
 - par exemple, un employé qui collabore avec d'autres employés.



Non hiérarchique



hiérarchique

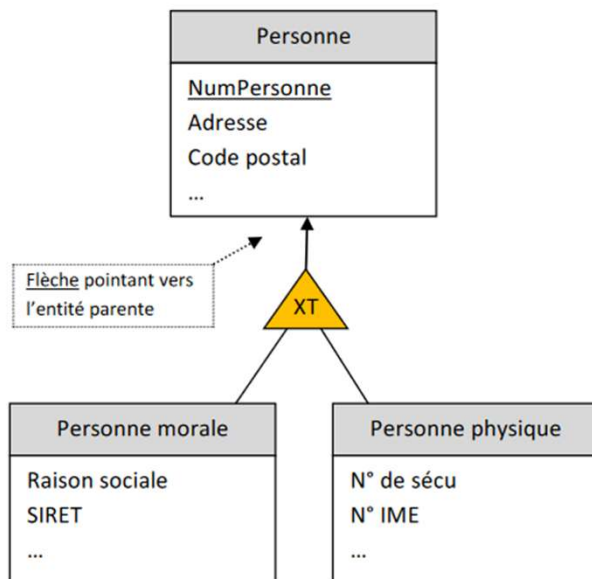




CAS PARTICULIER : L'HÉRITAGE

ÉLÉMENTS MCD

L'héritage



Contrainte sur la spécialisation et la généralisation

- **() vide** : tout et n'importe quoi. On est l'une, l'autre, les deux ou aucune
- **Exclusion (X)** : équivaut à un ou exclusif. On est l'une, l'autre ou aucune mais pas les deux à la fois.
- **Partition (XT)** : totalité + exclusivité. On est l'une, l'autre (et donc ni les deux, ni aucune des deux).
- **Totalité : (T)** équivaut à un ou logique. On est l'une, l'autre, les deux mais pas aucune des deux.

Pour l'une ou l'autre = les entités filles !

POURQUOI REPRÉSENTER L'HÉRITAGE CONCEPTUELLEMENT ?

- Clarté sémantique :
 - L'héritage permet de modéliser des relations "est-un"
 - ex : un Étudiant est une Personne, ce qui rend le modèle plus lisible et proche du langage métier.
- Réutilisation des attributs :
 - Éviter la redondance
 - ex : nom, prénom, date_naissance sont définis une seule fois dans Personne et hérités par Étudiant.
- Flexibilité :
 - Facilite l'ajout de sous-types ultérieurement
 - ex : Enseignant, Administratif.

Conséquences techniques :

La plupart des SGBD ne gèrent pas l'héritage de tables.

Il faut donc le "simuler" avec des solutions comme :

- Une table par classe concrète (TPC) :
 - Chaque sous-type devient une table indépendante avec tous les attributs (redondance possible).
- Une table par classe (TPH) :
 - Une seule table avec une colonne discriminante (ex : type = "étudiant") et des colonnes nulles pour les attributs spécifiques.
- Une table par sous-type (TPS) :
 - Une table pour la super-classe + une table par sous-classe (liées par clé étrangère).

Conclusion :

L'héritage dans le MCD est utile pour la clarté, mais il faut anticiper sa traduction technique.

Choisissez la solution physique en fonction du SGBD et des besoins :

- Privilégiez TPS pour éviter la redondance.
- Évitez TPH si beaucoup de colonnes sont nulles.

Documenter les choix :

pour expliquer pourquoi une table "pour rien" existe (ex : table de lien pour TPS).

Sur les attributs

EVITER LA REDONDANCE
D'INFORMATIONS GRÂCE À LA
DÉPENDANCE FONCTIONNELLE ^{DF}

ÉVITER LA REDONDANCE D'INFORMATIONS

La redondance, c'est problématique, pour différentes raisons.

- Premièrement cela peut provoquer des incohérences dans les données.
- Ensuite, Il faudra actualiser l'information à plusieurs endroits et on aura vite fait d'oublier une ou deux lignes : cela créera encore des incohérences !

Alors quelle est la solution ?

- Il faut quand cela est pertinent mettre en place une indépendance entre les données.
 - Créer une entité propre pour une information qui sera plus facile à contrôler, mettre à jour par exemple.
 - En bonus, gain de place en mémoire !!
 - Ce sera l'identifiant de l'entité nouvellement créée qui permettra le lien avec les autres entités.

En résumé :

- Quand une information est présente plusieurs fois dans une base de données, on dit qu'il y a redondance.
- Une redondance dans un tableau de données peut provoquer des incohérences dans les données.
- Selon la modélisation relationnelle, le moyen d'y remédier est de séparer ce tableau en deux tableaux, liés entre eux par une clé étrangère.

EXEMPLE

Dans un modèle conceptuel de site de ventes en ligne, nous avons une information concernant l'adresse du client.

Celle-ci est utile pour :

- Son adresse personnel
- Son adresse de facturation
- Son adresse de livraison

Ce qui donnerait :

- Client (nom, prénom, adresse du client)
- Facture (identifiant, adresse du client)
- Commande (identifiant, adresse du client)

La redondance se produit lorsqu'une même information est stocké plusieurs fois dans la base de données.

Ici, cela donnerait :

- Une entité Commande avec l'adresse
- Une entité Facture avec l'adresse
- Une entité Livraison avec l'adresse

Problèmes :

- Si l'adresse change alors je dois mettre à jour l'ensemble des entités où elle apparaît.
 - Gaspi d'espace mémoire
 - Difficulté de maintenance.

Pour l'éviter, il faut bien identifier les entités et leur association.

- Une entité client contiendra les informations du client dont l'adresse
- Une entité commande contiendra les informations de la commande dont l'identité du client
- Une entité facture contiendra les informations de la facturation dont l'identité de la commande concernée
- Une association indiquera la relation entre les entités : le client passe une commande. Une commande comprend une facture
- En cas de changement d'adresse, seule l'entité client sera mise à jour.

DÉPENDANCES FONCTIONNELLES (DF)

Les dépendances fonctionnelles structurent les attributs exprimant qu'un attribut ou ensemble d'attributs A détermine un autre attribut ou ensemble d'attributs B au sein d'une entité.

- Cela se traduit par $A \rightarrow B$, ce qui signifie que pour chaque valeur de A, il n'existe qu'une seule valeur de B.



Points clés à mentionner :

- Les DF permettent d'identifier les relations entre attributs dans un modèle conceptuel des données (MCD).
- Elles sont cruciales pour normaliser les bases de données, en réduisant la redondance et en évitant les anomalies.

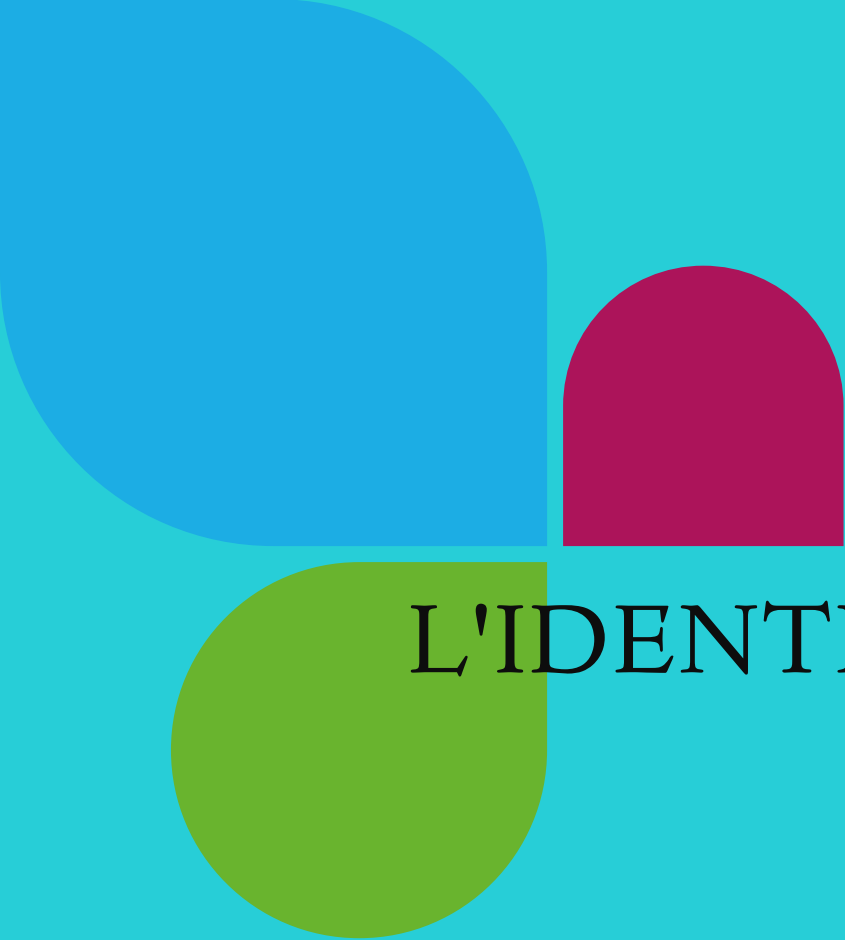
- Si un employé est identifié par un identifiant unique `ID_Employé`, alors toutes ses informations personnelles (nom, prénom, date de naissance, etc.) dépendent fonctionnellement de cet ID.

- On peut représenter cela comme :

- $A \rightarrow B$
- $\{ID_Employé\} \rightarrow \{Nom, Prénom, Date\ de\ naissance\}$



Souvenez-vous : ce sont ces dépendances fonctionnelles que vous avez déterminés lors de votre analyse du dictionnaire des données.



L'IDENTIFIANT RELATIF

UTILISATION D'UN IDENTIFIANT RELATIF (1,1) OU 1,1 (R)

Quoiqu'une entité soit une unité a priori indépendante, il arrive qu'on veuille la faire dépendre d'une ou plusieurs entités. En ce sens, une telle entité s'apparente à une association.

Il y a principalement deux cas pratiques où l'on est amené à faire dépendre une entité d'une ou plusieurs autre(s) :

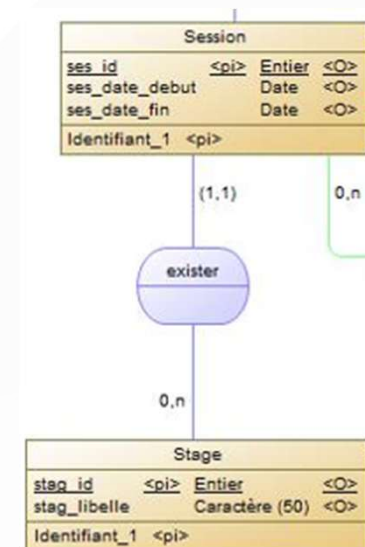
Cas n° 1 : lorsque l'on souhaite que l'identifiant d'une entité dépende d'une autre entité.

- Exemple : faire dépendre l'identifiant de la facture avec l'identifiant de la commande par exemple

Cas n° 2 : lorsque l'existence d'une entité n'a de sens que sous réserve de l'existence d'une ou plusieurs autres.

- Exemple : un appartement n'existe que sous réserve de l'existence de l'immeuble. Détruisez l'immeuble et les appartements seront détruits.

- Une **DF forte** si un lien existe à tout moment pour l'ensemble des occurrences de l'association. (1,1)
- Une **DF faible** sinon (0,1)





EXERCICES

EXERCICES METIS

Exercices METIS

Effectuer les exercices du document :

- 04-P-MCD.pdf



LES CONTRAINTES D'ASSOCIATIONS : CONTRAINTES D'INTÉGRITÉ RÉFÉRENTIELLES

CONTRAINTES D'INTÉGRITÉ RÉFÉRENTIELLES

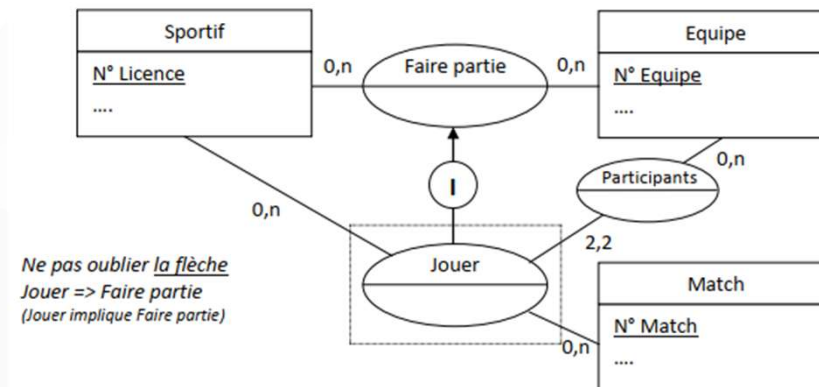
CONTRAINTE D'INCLUSION

- Signification : Si un élément participe à une relation A, alors il doit aussi participer à une relation B.

L'ensemble des couples (SPORTIF, EQUIPE) qui participent à l'association "JOUER" est inclus dans l'ensemble de ceux qui participent à l'association "FAIT PARTIE".

Autrement dit : tout sportif qui joue un match avec une équipe doit faire partie de cette équipe.

Exemple



CONTRAINTES D'INTÉGRITÉ RÉFÉRENTIELLES

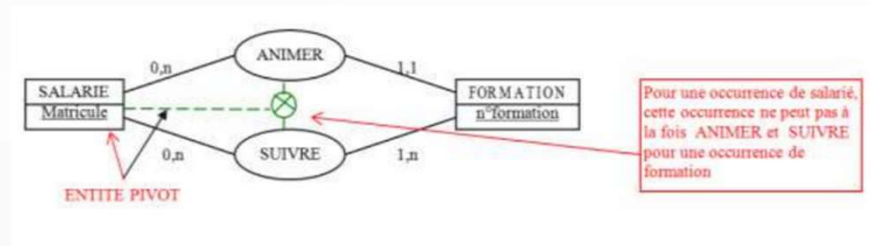
CONTRAINTE D'EXCLUSION (X)

- Signification : Un élément ne peut pas participer simultanément aux deux relations. Il peut participer à l'une, à l'autre, ou à aucune, mais pas aux deux en même temps.

Au sein d'une entreprise, des formations sont organisées. Un salarié anime la formation, d'autres salariés suivent la formation.

N.B.: les traits en pointillés sont appelés les pivots. Le ou les pivot(s) pointent les entités dont la clef primaire doit figurer à la fois dans la ou les association(s) « sources » et dans l'association de « destination ».

Exemple



CONTRAINTES D'INTÉGRITÉ RÉFÉRENTIELLES

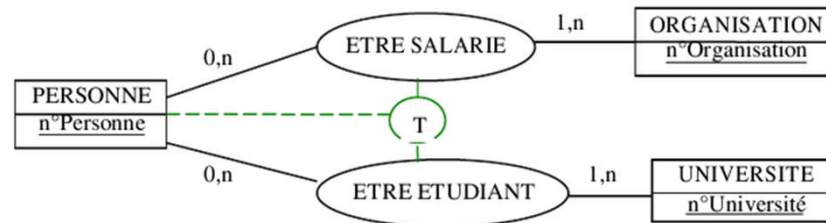
CONTRAINTE DE TOTALITÉ (T)

- Signification : Un élément doit obligatoirement participer à au moins une des deux relations. Il peut participer à l'une, à l'autre, ou aux deux.

Pour une personne, elle est soit étudiante, soit salarié, soit les deux.

Toute occurrence du pivot participe forcément à une des deux associations, voire aux deux.

Exemple



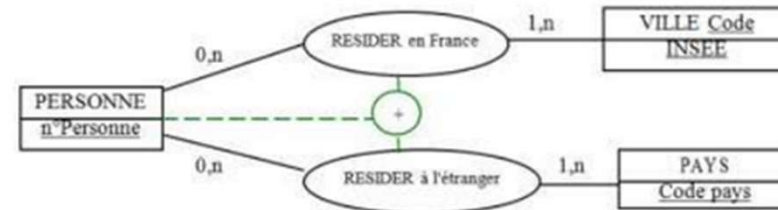
CONTRAINTES D'INTÉGRITÉ RÉFÉRENTIELLES

CONTRAINTE DE PARTITION (XT OU +)

- Signification : Un élément doit participer à une et une seule des deux relations. Il ne peut pas participer aux deux en même temps, ni à aucune.

Une personne réside forcément en France ou à l'étranger, mais pas dans les deux.

Exemple



CONTRAINTES D'INTÉGRITÉ RÉFÉRENTIELLES

CONTRAINTE DE SIMULTANÉITÉ (=)

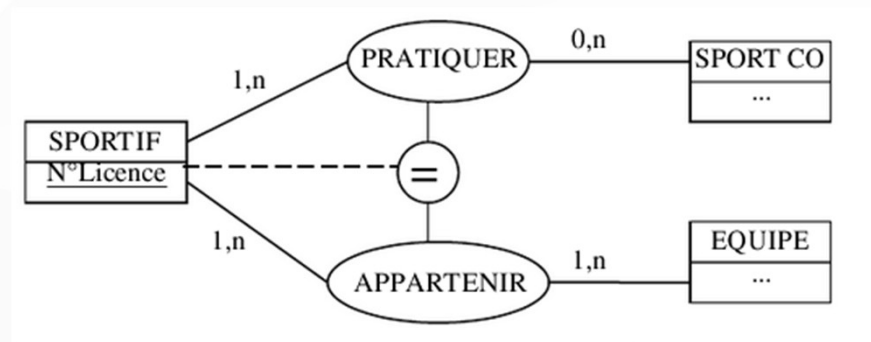
- Signification : Un élément doit participer aux deux relations en même temps. C'est un « ET » logique.

Tout sportif qui pratique un sport collectif fait partie d'une équipe et vice-versa.

C'est une contrainte d'inclusion dans les deux sens

On a autant d'occurrence de sportifs qui participe à "PRATIQUER" que à "APPARTENIR". Un sportif S1 participe à autant d'associations dans "PRATIQUER" et "APPARTENIR".

Exemple



Règles métiers

CONTRAINTE D'INTEGRITÉ FONCTIONNELLE CIF

CONTRAINTE D'INTÉGRITÉ FONCTIONNELLE (CIF)

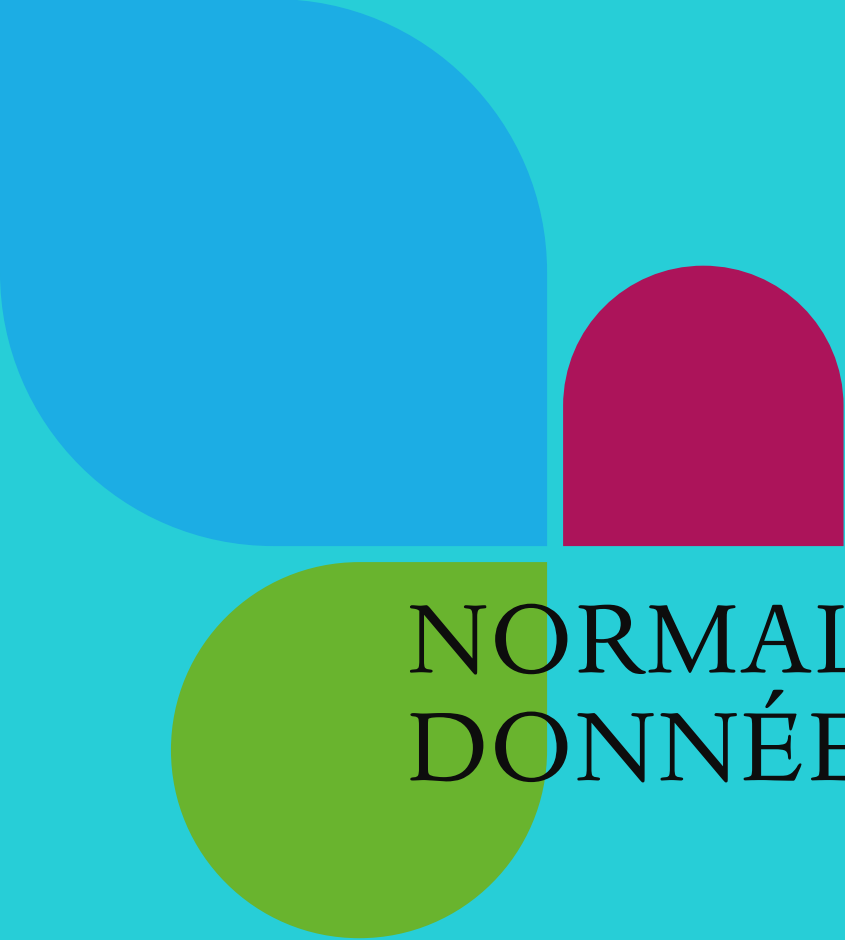
Une Contrainte d'Intégrité Fonctionnelle (CIF) est **une règle métiers** qui garantit la cohérence des données en s'assurant qu'une dépendance fonctionnelle est toujours respectée dans la base de données.

- Si $X \rightarrow Y$ est une dépendance fonctionnelle, alors pour chaque valeur de X, il ne peut y avoir qu'une seule valeur de Y **et que cette relation doit toujours être vraie.**
- Exemple :
 - Dans une table Client, la dépendance fonctionnelle $\{id_client\} \rightarrow \{nom_client\}$ signifie qu'un identifiant de client ne peut être associé qu'à un seul nom de client.
 - Si cette règle est violée (ex : deux noms différents pour le même `id_client`), la base de données devient incohérente.

Rôle de la CIF dans Merise :

- **Garantir la qualité des données** : Éviter les incohérences et les anomalies (ex : doublons, valeurs contradictoires).
- **Respecter les règles métiers** : Traduire les contraintes du monde réel dans le modèle de données.
- **Faciliter la normalisation** : Aider à atteindre les formes normales (1FN, 2FN, 3FN) en identifiant les dépendances fonctionnelles.

• Une CIF est une DF qui est à la fois forte, stable dans le temps, et imposée comme règle inviolable par le métier. Dans une table Produit, $code_produit \rightarrow prix_unitaire$ peut être une DF, mais si le prix est fixé par contrat et ne peut pas changer arbitrairement, cela devient une CIF.



NORMALISATION DES DONNÉES

NORMALISATION DES DONNÉES

Dans une base de données relationnelle, une forme normale désigne un type de relation particulier entre les entités.

Comme son nom l'indique, normale décrit la norme.

La normalisation des modèles de données permettra donc :

- de s'assurer de la robustesse de la conception de la base pour améliorer la modélisation (et donc obtenir une meilleure représentation)
- optimiser la mémorisation des données en évitant la redondance et les problèmes sous-jacents de mise à jour ou de cohérence.

La normalisation s'applique à toutes les entités et aux relations porteuses de propriétés.

On dira d'un modèle qu'il est en première, deuxième ou troisième forme normale :

- *Il s'agit là des règles de normalisation les plus importantes, d'autres règles pouvant être appliquées mais de manière non systématique.*

La forme normale vient après la simple validité d'un modèle relationnel, c'est-à-dire que les valeurs des différents attributs soient bien en dépendance fonctionnelle avec la clé primaire (complètement déterminés par la clé primaire).

En pratique, la première et la deuxième forme normale sont nécessaires pour avoir un modèle relationnel juste.

- Les formes normales supplémentaires ont leurs avantages et leurs inconvénients.
- La troisième forme normale reste généralement une des meilleures solutions d'un point de vue architecture de base de données, mais pour des bases de données plus importantes, cela n'est pas toujours le cas.

NORMALISATION DES DONNÉES

Les avantages

1. de limiter les redondances de données
2. de limiter les incohérences de données qui pourraient les rendre inutilisables
3. d'éviter les processus de mise à jour récurrents

Les inconvénients

1. des temps d'accès qui peuvent être plus longs si les requêtes sont trop complexes
2. un manque de flexibilité au niveau de l'utilisation de l'espace de stockage

PREMIÈRE FORME NORMALE (NOTÉE 1FN)

Respecte la première forme normale, la relation dont **tous les attributs** :

- contiennent **une valeur atomique** (les valeurs ne peuvent pas être divisées en plusieurs sous-valeurs dépendant également individuellement de la clé primaire).
 - *Ainsi, le nom et le prénom d'une personne doivent être dans des propriétés distinctes.*
- contiennent **des valeurs non répétitives** (le cas contraire consiste à mettre une liste dans un seul attribut).
 - *On pourrait citer pour exemple une catégorie d'articles qui doit être définie dans une entité spécifique et non répétée dans l'entité article.*
- sont **constants dans le temps**
 - *utiliser par exemple la date de naissance plutôt que l'âge.*

Le non-respect de deux premières conditions de la 1FN rend la recherche parmi les données plus lente parce qu'il faut analyser le contenu des champs. La troisième condition quant à elle évite de régulièrement mettre à jour les données.

EXEMPLES D'ERREURS

Quelques exemples de violation

1FN — première forme normale [\[modifier \]](#) [\[modifier le code \]](#)

- tout attribut contient une valeur atomique

<u>CLIENT_ID</u>	...
Dupont Paris	
Durand Marseille	

L'attribut CLIENT_ID est composé de 2 attributs atomiques.

<u>CLIENT_ID</u>	NOM
1	Gérard Dupont
2	Léon Durand

L'attribut NOM est composé de 2 attributs atomiques.

- tous les attributs sont non répétitifs

<u>PRODUIT_ID</u>	DESCRIPTION	FOURNISSEURS
1	Téléviseur	Sony, Sharp, LG

L'attribut FOURNISSEURS est une liste.

- tous les attributs sont constants dans le temps.

<u>CLIENT_ID</u>	NOM	PRENOM	AGE
1	Dupont	Gérard	35

L'attribut AGE n'est pas constant dans le temps.

DEUXIÈME FORME NORMALE (NOTÉE 2FN)

Respecte la seconde forme normale, la relation respectant la première forme normale et dont :

- Les attributs d'une relation sont divisés en deux groupes :
 - le premier groupe est composé de l'identifiant (un ou plusieurs attributs).
 - Le deuxième groupe est composé des autres attributs (éventuellement vide).
- La deuxième forme normale stipule que tout attribut du deuxième groupe ne peut pas dépendre d'un sous-ensemble (strict) d'attribut(s) du premier groupe.
- En d'autres termes :
 - Un attribut non identifiant ne dépend pas d'une partie de l'identifiant mais de tout l'identifiant.
 - Tous les attributs non identifiants doivent dépendre de toute la clé primaire (et non d'une partie seulement).
- Le non-respect de la 2FN entraîne une redondance des données qui encombrant alors inutilement la mémoire et l'espace disque.

☹️ $R (\underline{a1}, \underline{a2}, \underline{a3}, a4, a5, a6)$

TROISIÈME FORME NORMALE (NOTÉE 3FN)

Respecte la troisième forme normale, la relation respectant la seconde forme normale et dont :

- La troisième forme normale stipule que tout attribut du deuxième groupe ne peut pas dépendre d'un sous-ensemble (strict et excluant l'attribut considéré) d'autres attribut(s) du second groupe.
- En d'autres termes : « Un attribut non identifiant ne dépend pas d'un ou plusieurs attributs ne participant pas à l'identifiant ».
- Le respect de la 3FN ne garantit pas une absence de redondance des données d'où la FNBC.



$R (\underline{a1}, \underline{a2}, \underline{a3}, a4, a5, a6)$



EXEMPLES D'ERREURS

2FN — deuxième forme normale [\[modifier \]](#) [\[modifier le code \]](#)

- tous les attributs non identifiants sont totalement dépendants fonctionnellement de la totalité de l'identifiant (et donc de la clef primaire au niveau tabulaire).

<u>COMMANDE_ID</u>	<u>ARTICLE_ID</u>	DESCRIPTION_ARTICLE
1	15	TV haute définition avec amplificateur dolby 5.1

L'attribut DESCRIPTION_ARTICLE ne dépend que d'une partie de la clef primaire.

3FN — troisième forme normale [\[modifier \]](#) [\[modifier le code \]](#)

- tout attribut n'appartenant pas à un identifiant ne dépend pas d'un attribut non identifiant

<u>COMMANDE_ID</u>	<u>CLIENT_ID</u>	NOM_CLIENT
1	1	Durand

L'attribut NOM_CLIENT dépend de CLIENT_ID.

FNBC : FORME NORMALE DE BOYCE-CODD

Il s'agit d'une version plus stricte de la 3^{ème} forme normale.

Si une entité ou une relation en troisième forme normale a un identifiant composé, aucune des propriétés élémentaires de cet identifiant ne doit être en dépendance fonctionnelle d'une autre propriété.

- Le non-respect de la 2FN, 3FN et la FNBC entraîne de la redondance, une même donnée étant répétée un nombre considérable de fois.

Contexte :

Supposons une base de données pour gérer les cours dans une université.

Nous avons une table Cours_Prof_Salle avec les attributs suivants :

Professeur	Cours	Salle
Dupont	Bases de données	A104
Martin	Algèbre	B202
Dupont	Réseaux	A104

Problème :

Dans cette table, il existe une dépendance fonctionnelle : Professeur → Salle

- Chaque professeur a une salle attitrée (ex. : Dupont enseigne toujours en A104).

Cependant, Professeur n'est pas une clé candidate de la table. La clé candidate ici est probablement {Professeur, Cours} (car un professeur peut enseigner plusieurs cours, mais pas dans des salles différentes).

FNBC : FORME NORMALE DE BOYCE-CODD

Anomalies possibles : Redondance :

- La salle A104 est répétée pour Dupont.

Anomalie de mise à jour :

- Si Dupont change de salle, il faut mettre à jour plusieurs lignes.

Anomalie d'insertion :

- Impossible d'ajouter un professeur sans lui attribuer un cours.

Anomalie de suppression :

- Si on supprime le cours "Bases de données", on perd l'information que Dupont enseigne en A104.

Solution : Décomposer en FNBC Pour respecter la FNBC, on décompose la table en deux :

Table 1 : Prof_Salle

Professeur	Salle
Dupont	A104
Martin	B202

Table 2 : Prof_Cours

Professeur	Cours
Dupont	Bases de données
Dupont	Réseaux
Martin	Algèbre

Résultat : Chaque dépendance fonctionnelle (Professeur $\rightarrow$ Salle) a maintenant un déterminant (Professeur) qui est une clé candidate dans sa table. Plus de redondance ni d'anomalies.

FNDC : FORME NORMALE DOMAINE CLÉ

Une relation est en FNDC, si et seulement si toutes les contraintes sont la conséquence logique des contraintes de domaines et des contraintes de clefs qui s'appliquent à la relation.

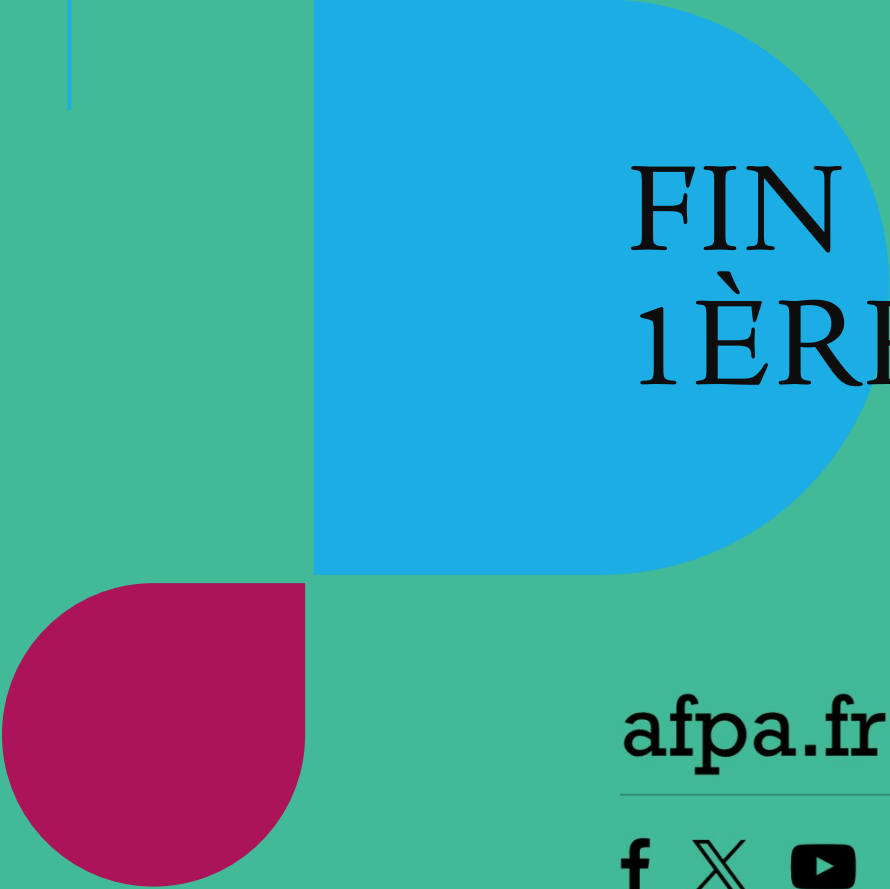
Pour se souvenir de l'ordre et des caractéristiques des trois premières formes normales, il suffit de se rappeler le serment que tous les témoins doivent prêter devant la justice : *Je jure de dire la vérité, toute la vérité, rien d'autre que la vérité.*

Ce qui donne : 1FN = La clef. 2FN = Toute la clef. 3FN = Rien que la clef.

Soit la relation VEHICULE, avec les attributs suivants :

<u>CONSTRUCTEUR</u>	<u>MODELE</u>	<u>TYPE</u>	<u>PTAC (KG)</u>
Renault	Estafette	VL	2500
Iveco	Eurostar 440	PL	19000
Berliet	GDM 1934	PL	15000
Volkswagen 2 900	combi	VL	2 900

- *On remarque que le type VL (véhicule léger) ou PL (poids lourd) est déterminé par la valeur du PTAC.*
- *Il y a redondance des données de type qui peut être déduite de la lecture de la valeur du PTAC.*
- *Pour résoudre cette anomalie de mise à jour, il faut décomposer la relation en deux soit une entité Véhicule et une entité Type*



FIN 1ÈRE PARTIE

afpa.fr



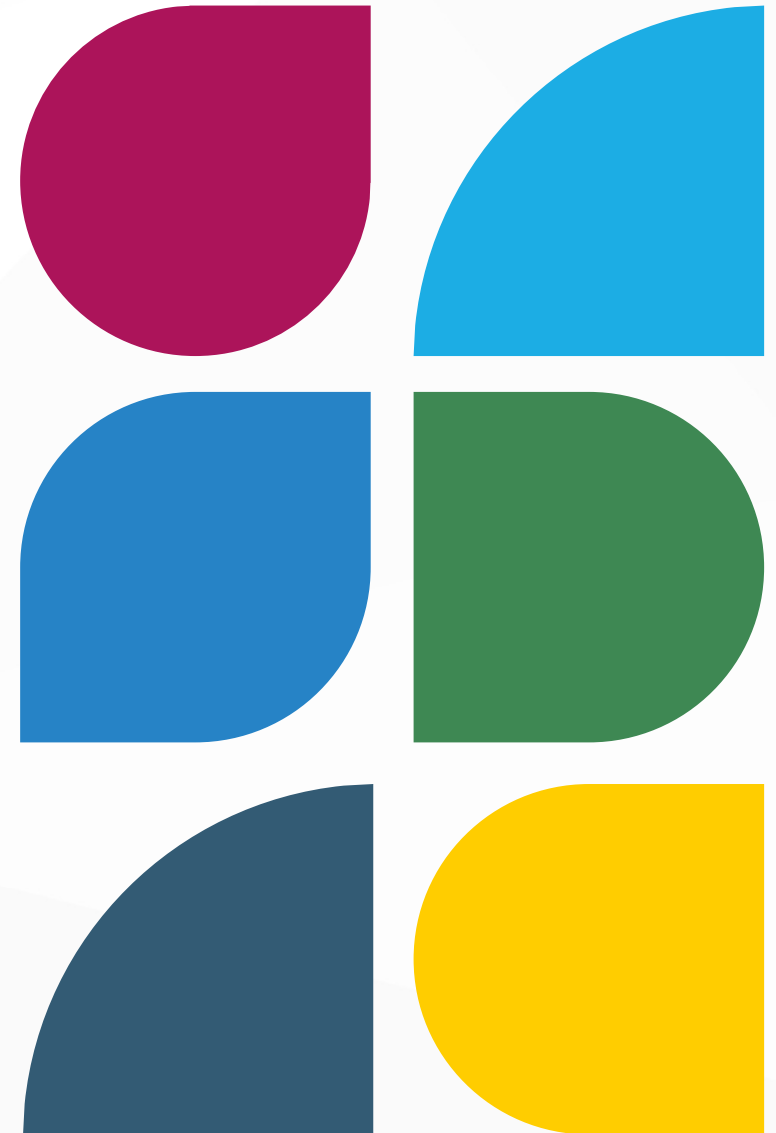
Jérôme BOEBION

Version 4 - révision septembre 2025

LA MÉTHODE MERISE

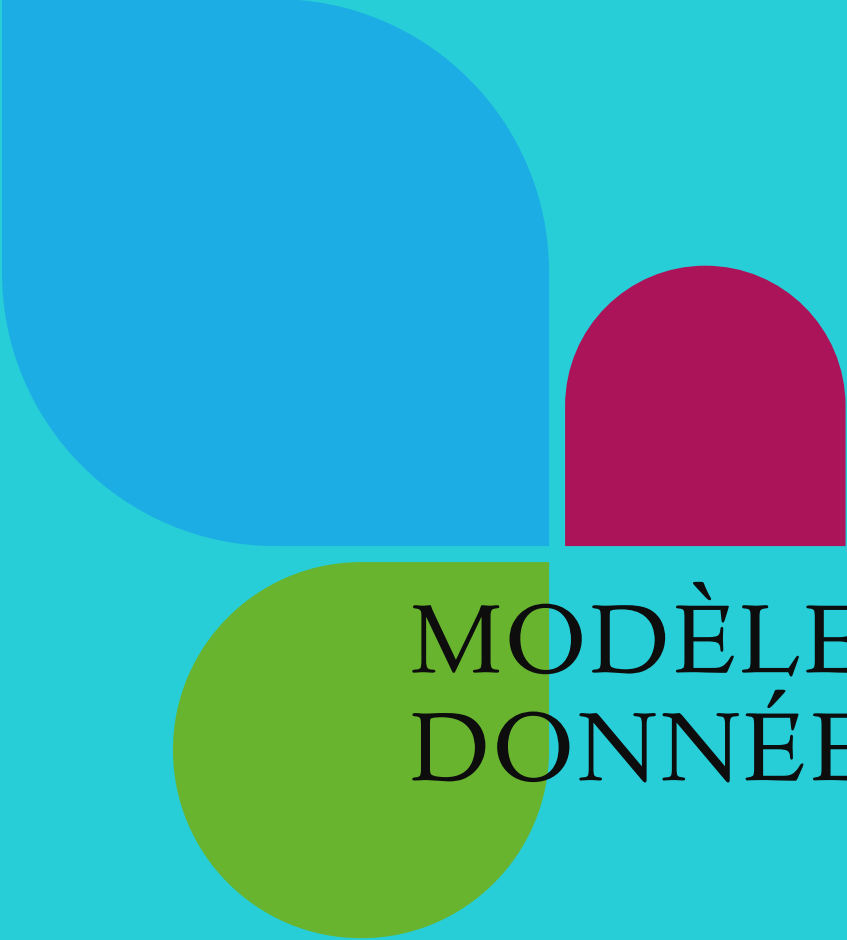
Concevoir et développer la persistance de données.
- Partie 2 -

Concepteur Développeur d'Applications





ETAPE 4-5 - MODEL LOGIQUE ET PHYSIQUE DES DONNÉES.



MODÈLE LOGIQUE DES DONNÉES.

MODÈLE LOGIQUE DES DONNÉES MLD

Le vocabulaire change à ce moment de la transformation.

- Les entités deviennent **des relations** c-a-d **des tables**.
- Les identifiants deviennent **des clés**
- Les relations **disparaissent ou se transforment** en tables selon les cas, en transférant les clés des tables en fonction des cardinalités.

Le MLD représente l'expression du MCD mais précisant la volumétrie, la structure et l'organisation des données telles qu'elles pourront être implémentées.

MCD	Relation	SGBD
Entité	Relation	Table
Association	Relation	Table
Identifiant	Clé primaire	Clé primaire
	Clé étrangère	Clé étrangère
Attribut, propriété	Attribut	Champs, colonne
Occurence, instance	Occurence	Occurence, enregistrement

MODÈLE LOGIQUE DES DONNÉES MLD

C'est également dans ce modèle que nous allons affiner le typage des attributs pour définir leur type, leur taille etc...

En fait, le MLD découle du MCD.

Pas besoin de trop réfléchir : il n'y a qu'à appliquer des règles de transformation basiques pour passer du MCD au MLD.

En bref :

- La table (encore appelée 'relation'), concept central du modèle relationnel, peut être vue comme un tableau de données ;
- Les colonnes de ce tableau sont appelés les attributs de la relation ;
- Chaque attribut peut prendre des valeurs dans un domaine défini par le type des données ;
- Un ou plusieurs de ces attributs forme la clé d'accès aux lignes du tableau ; les valeurs prises par cette clé sont uniques dans une même table ;
- Les lignes du tableau, occurrence de la table, sont appelées enregistrements, tuples ou n-uplets ;
- Il est d'usage de donner des noms d'attributs qui ne soient pas des mots composés et qui ne contiennent pas de lettres accentuées.

On représente souvent une table par son '**schéma**' décrit **en extension** ou **graphiquement**. Il est d'usage de **souligner la clé** d'une table dans les différentes représentations.

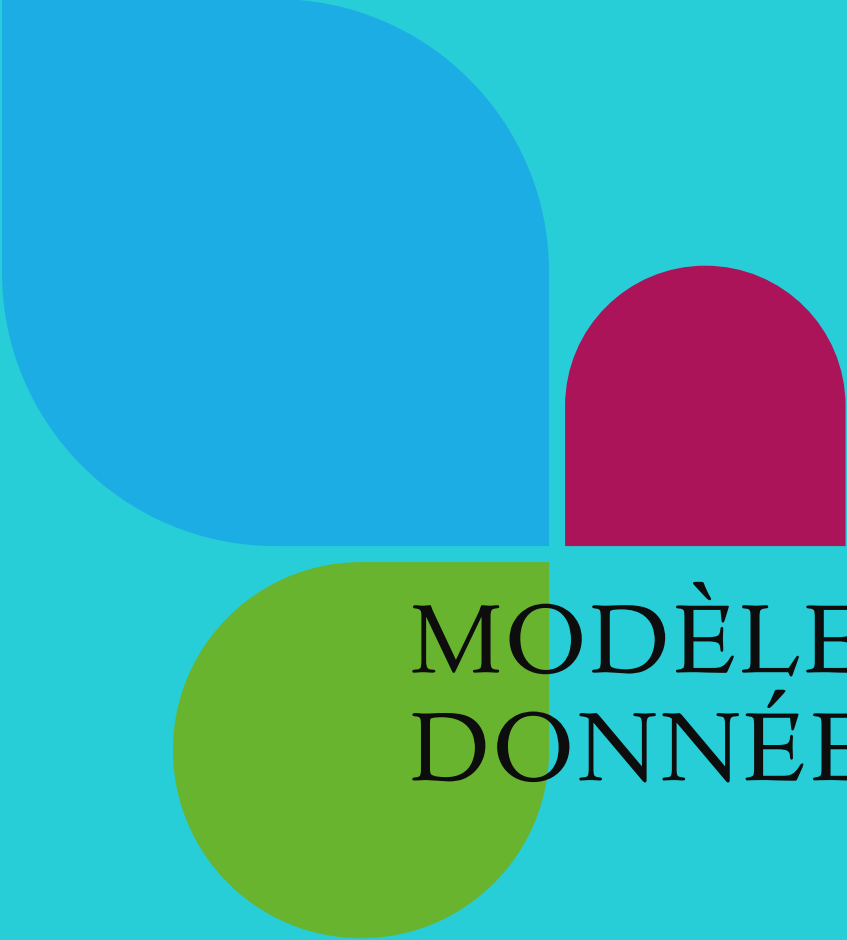
Ainsi dans notre exemple précédent, la table des contacts pourra être représentée en extension par :

Contacts (numero, civilite, nom, prenom, telfixe, telmobile, email)

complété par la liste des domaines/types de données des colonnes.

Ou encore, graphiquement par :

contacts	
numero	integer
civilite	char(5)
nom	char(25)
prenom	character(15)
telfixe	character(15)
telmobile	character(15)
email	character(50)



MODÈLE PHYSIQUE DES DONNÉES.

inclus la notion de l'implémentation dans le futur Système de Gestion de Bases de Données Relationnelles.

MODELE PHYSIQUE DES DONNEES - MPD

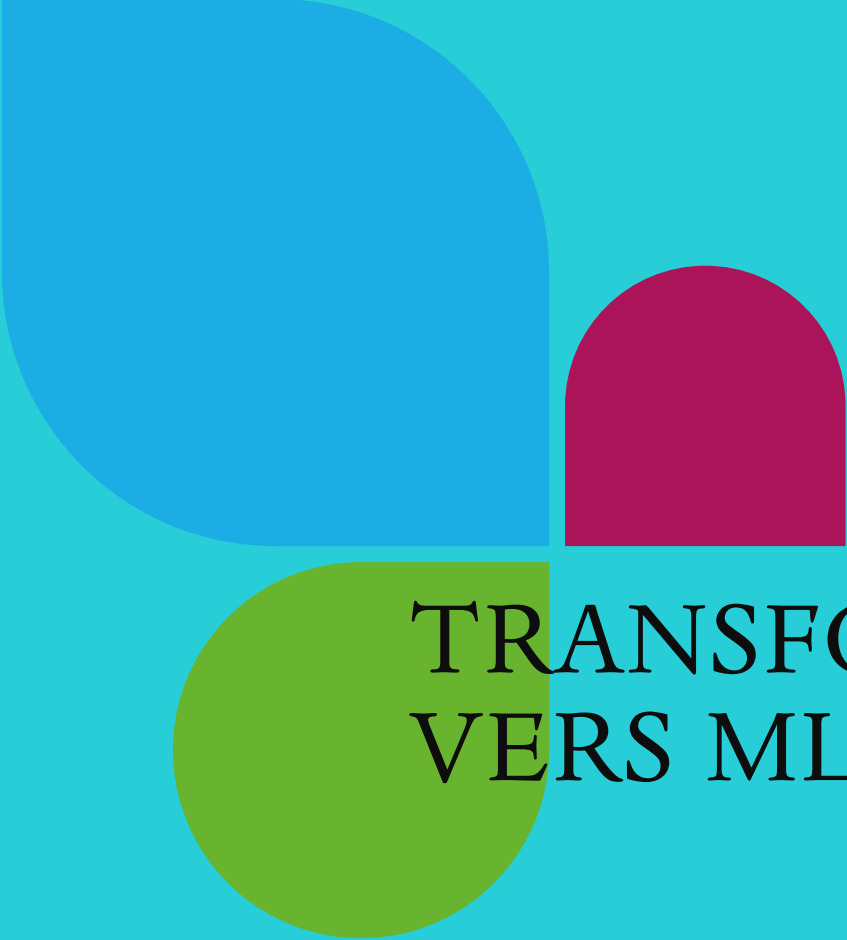
Étant très proche visuellement, on passe souvent du MCD au MPD directement.

le MPD servira donc à traduire le modèle relationnel en code SQL compréhensible par le SGBDR que vous aurez choisi (PostgreSQL, MySQL, Oracle, SQLite, etc.).

- *Même si ces SGBDR comprennent tous le langage SQL, il existe parfois des différences.*

C'est-à-dire que chaque SGBDR a ses propres variantes du langage SQL :

- Le langage de définition de données (LDD ou DDL)
 - pour créer la structure de la BDD
- Le langage de manipulation de données (LMD ou DML)
 - pour insérer, actualiser, supprimer ou récupérer des données
- Le langage de contrôle des données (LCD ou DCL)
 - pour gérer les droits d'accès aux données
- Le langage de contrôle des transactions (LCT ou TCL)
 - pour la validation ou l'annulation de modifications de données.



TRANSFORMATIONS MCD VERS MLD / MDP.

RÈGLES DE PASSAGE

La 1ère règle est très simple :

- Toutes entités deviennent une **table**.
- Son identifiant devient sa **clé primaire**.
- Une colonne correspond à **une propriété de la table**.
- Une ligne de la table est un **enregistrement**.

Règles de passage du MCD au MLD :

Règle numéro 1 :

a) Une entité du MCD devient une relation, c'est à dire une table.

Dans un SGBD de type relationnel, une table est structure tabulaire dont chaque ligne correspond aux données d'un objet enregistré (d'où le terme enregistrement) et où chaque colonne correspond à une propriété de cet objet. Une table contiendra donc un ensemble d'enregistrements.

Une ligne correspond à un enregistrement.

Une colonne correspond à un champ.

La valeur prise par un champ pour un enregistrement donné est située à l'intersection ligne-colonne correspondant à enregistrement-champ.

Il n'y a pas de limite théorique au nombre d'enregistrements que peut contenir une table. Par contre, la limite est liée à l'espace de stockage.

b) Son identifiant devient la clé primaire de la relation.

La clé primaire permet d'identifier de façon unique un enregistrement dans la table.

Les valeurs de la clé primaire sont donc uniques.

Les valeurs de la clé primaire sont obligatoirement non nulles.

Dans la plupart des SGBDR, le fait de définir une clé primaire donne lieu automatiquement à la création d'un index.

c) Les autres propriétés deviennent les attributs de la relation.

Exemple :

CLIENT
<u>numClient</u>
nom
prénom
adresse

CLIENT(numClient, nom, prénom, adresse)

RÈGLES DE PASSAGE

la 2^{ème} règle parle du transfert de l'identifiant de l'entité forte vers l'entité faible

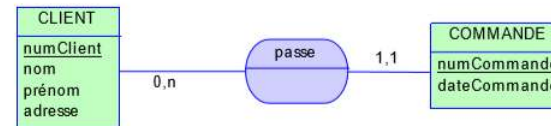
- Entité faible dont la valeur max de sa cardinalité est 1
- Entité forte dont la valeur max de sa cardinalité est n

Règle numéro 2 :

Une association de type 1:N (c'est à dire qui a les cardinalités maximales positionnées à « 1 » d'une côté de l'association et à « n » de l'autre côté) se traduit par la création d'une clé étrangère dans la relation correspondante à l'entité côté « 1 ».

Cette clé étrangère référence la clé primaire de la relation correspondant à l'autre entité.

Exemple :



CLIENT(numClient , nom , prénom , adresse)

COMMANDE(numCommande , dateCommande , #numClient)

RÈGLES DE PASSAGE

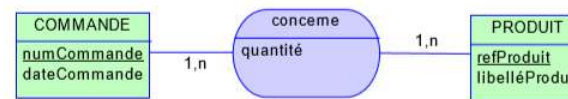
la 3^{ème} règle indique qu'une association reliée avec des entités ayant une cardinalité max n :

- transforme l'association en une table
- Le nom de la table sera le nom de l'association
- Sa clé primaire correspond à l'association des deux clés primaires des entités qui l'encadre.

Règle numéro 3 :

Une association de type N :N (c'est à dire qui a les cardinalités maximales positionnées à « N » des 2 côtés de l'association) se traduit par la création d'une relation dont la clé primaire est composée des clés étrangères référençant les relations correspondant aux entités liées par l'association.
Les éventuelles propriétés de l'association deviennent des attributs de la relation.

Exemple :



COMMANDE(numCommande ,dateCommande)
 PRODUIT(refProduit, libelleProduit)
 CONCERNE(#numCommande , #refProduit , quantité)

Si le nom du MCD n'est pas significatif, on peut renommer le nom de la table.
 Dans notre exemple, plutôt que la table « CONCERNE », on la nommera « LIGNE_DE_COMMANDE ».

LIGNE_DE_COMMANDE (#numCommande , #refProduit , quantité)

Associations ternaires : Les règles définies ci-dessus s'appliquent aux associations ternaires.

Associations réflexives : Les règles définies ci-dessus s'appliquent aux associations réflexives.

Exemple :



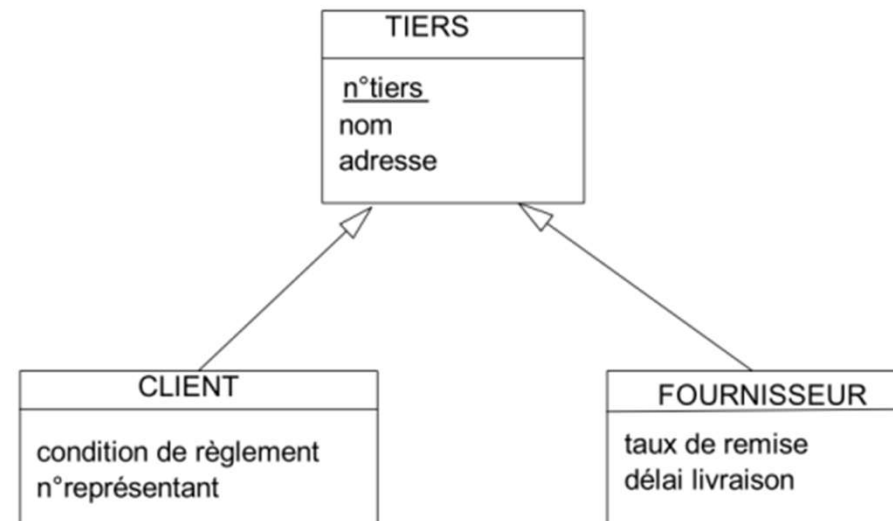
PIECE(codePiece ,libellePiece)
 COMPOSITION(refProduit, libelleProduit)
 COMPOSITION (#pieceComposee , #pieceComposante)

RÈGLES DE PASSAGE

La Spécialisation

- la clé primaire correspondant au sous-types (Client et Fournisseur) est celle de la table du sur-type (Tiers) de manière à réaliser le lien entre les enregistrements des tables.

Exemple :



Devient, en schéma relationnel :

```
TIERS (notiers, nom, adresse)
CLIENT (notiers, conditionReglement, noRepresentant)
FOURNISSEUR (notiers, tauxRemise, delaiLivraison)
```

RÈGLES DE PASSAGE

La généralisation

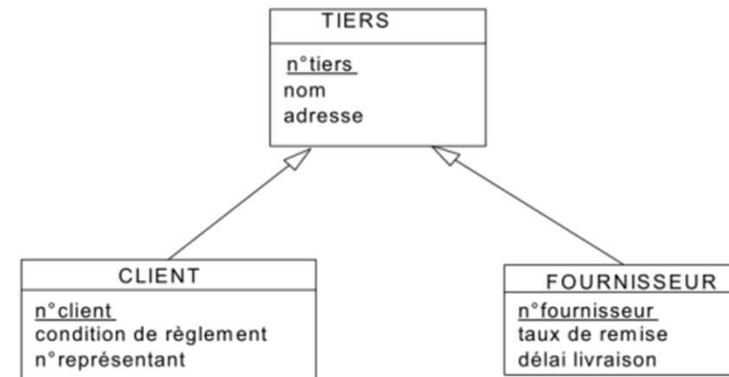
- Chaque sous-type est doté de sa propre clé primaire et reçoit la clé primaire du sur-type de manière à réaliser le lien entre les enregistrements des tables.

Schéma relationnel :

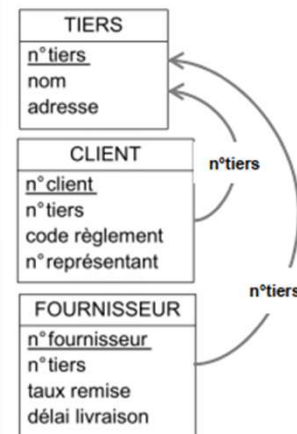
TIERS (notiers, nom, adresse)

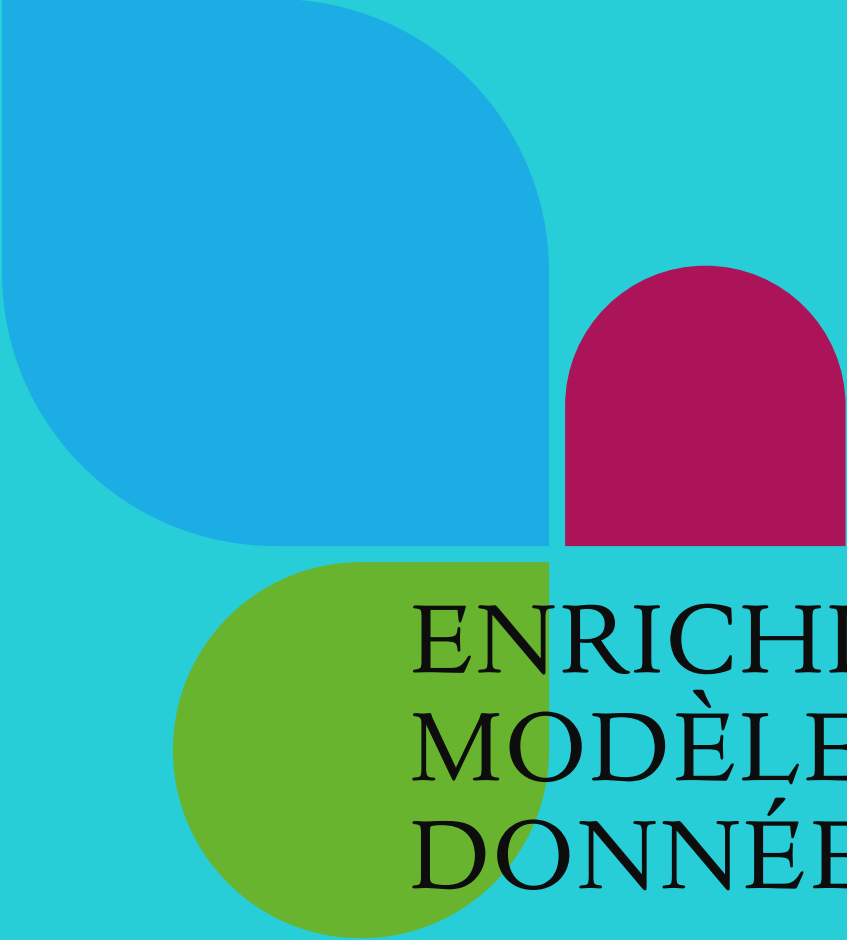
CLIENT (noclient, notiers, conditionReglement, noReprésentant)

FOURNISSEUR (nofournisseur, notiers, tauxRemise, delaiLivraison)



devient :





ENRICHISSEMENT DU MODÈLE PHYSIQUE DE DONNÉES

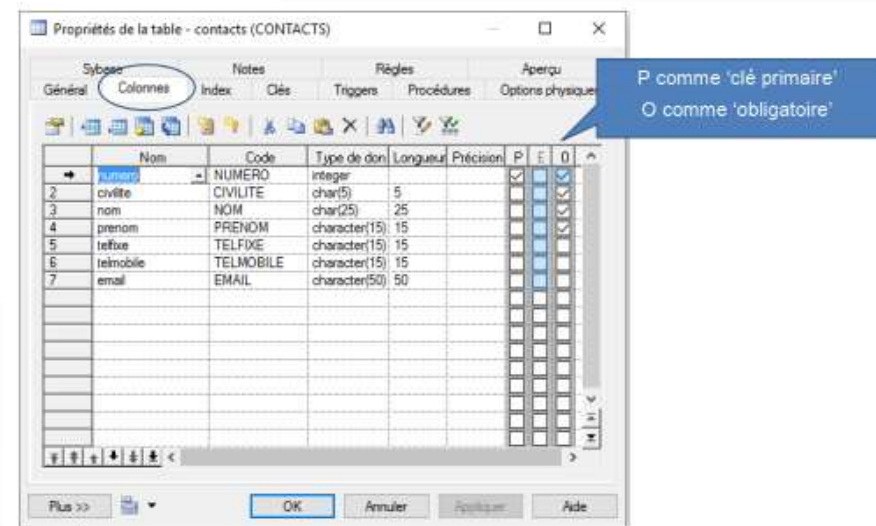
POSITIONNEMENT DES CONTRAINTES

C'est à partir de ce modèle que nous allons commencer à indiquer nos contraintes sur nos attributs.

Cela va composer la structure de la « table ».

Chaque « table » doit posséder une « clé » « unique » devant être courte et de longueur fixe.

Les contraintes sur les données par exemple « obligatoire ou facultative ».



ENRICHIR LE MPD

Les transformations décrites précédemment assurent un MPD Relationnel sans erreur que tout SGBD/R pourra implémenter.

On peut encore affiner le MPD de manière à sécuriser les données et optimiser les accès :

Index de table : on peut créer des index à partir des attributs d'une table (ex : nom et prénom d'une personne).

[Attention : non indispensable car consommateur d'espace mémoire et alourdit la BDD]

Contrainte de clé primaire (index de type unique)

- Accès immédiat à l'enregistrement,
- Refuse toute insertion dont la valeur de clé est déjà présente dans la table,
- Permet un tri des enregistrements sur les valeurs de la clé (tri croissant par défaut)

Exemple avec Power AMC :

The image displays three screenshots from the Power AMC database design tool, illustrating the configuration of a table and its indexes.

Top Screenshot: Table Properties for 'Salaries'

- The 'Notes' tab is selected, showing a list of attributes: Matricule (character(8)), INSEE (character(15)), Nom (character(25)), Prenom (character(25)), email (character varying(50)), rue (character(32)), complement (character(32)), codepostal (character(5)), ville (character(25)), and salairebrut (numeric(8,2)).
- A blue callout box points to the 'Matricule' attribute, stating: "La table et sa contrainte de clé primaire (Primary Key)".

Middle Screenshot: Key Properties for 'pkMatricule'

- The 'Properties de la clé - pkMatricule (PKMATRICULE)' dialog is shown.
- The 'Table' field is set to 'Salaries'.
- The 'Contrainte' field is set to 'PK-SALARIES'.
- The 'Clé primaire' checkbox is checked.

Bottom Screenshot: Index Properties for 'INSEE (INSEE)' and 'Nom (NOM)'

- The 'Properties de l'index - INSEE (INSEE)' dialog shows the 'Index' tab with 'INSEE' selected as the index name and 'INSEE' as the column.
- The 'Properties de l'index - (Nom) (NOM)' dialog shows the 'Index' tab with '(Nom)' selected as the index name and 'NOM' and 'PRENOM' as the columns.
- A blue callout box points to these two index configuration windows, stating: "3 index complémentaires".

ENRICHIR LE MPD

On peut encore affiner le MPD de manière à sécuriser les données et optimiser les accès.

Contraintes de clés étrangères :

- Génération d'un index supplémentaire qui impose de s'assurer de la présence de la clé dans la table de référence.

Contraintes de contrôle et valeurs par défaut :

Pour s'assurer de la cohérence, l'intégrité des données, les SGBD/R proposent des fonctionnalités diverses :

- Liste de valeurs associés à un attribut,
- Fourchette de valeurs associée à un attribut,
- Valeur par défaut,
- Domaine (ex : code postal sur 5 caractères compris en 01000 et 99999).

Toutes ces contraintes peuvent être initiées dès le MCD qui seront ensuite propagées au MPD par l'outil de modélisation.

Appliquées à la
base de données

CONTRAINTE D'INTÉGRATION MULTIPLES ^{CIM}

LA CONTRAINTE D'INTÉGRATION MULTIPLES (CIM)

Définition : Une CIM définit des règles pour maintenir la cohérence des données lors des opérations de mise à jour (insertion, suppression, modification).

Exemple : Si un client est supprimé, que faire de ses commandes ? (Suppression en cascade, mise à null, etc.).

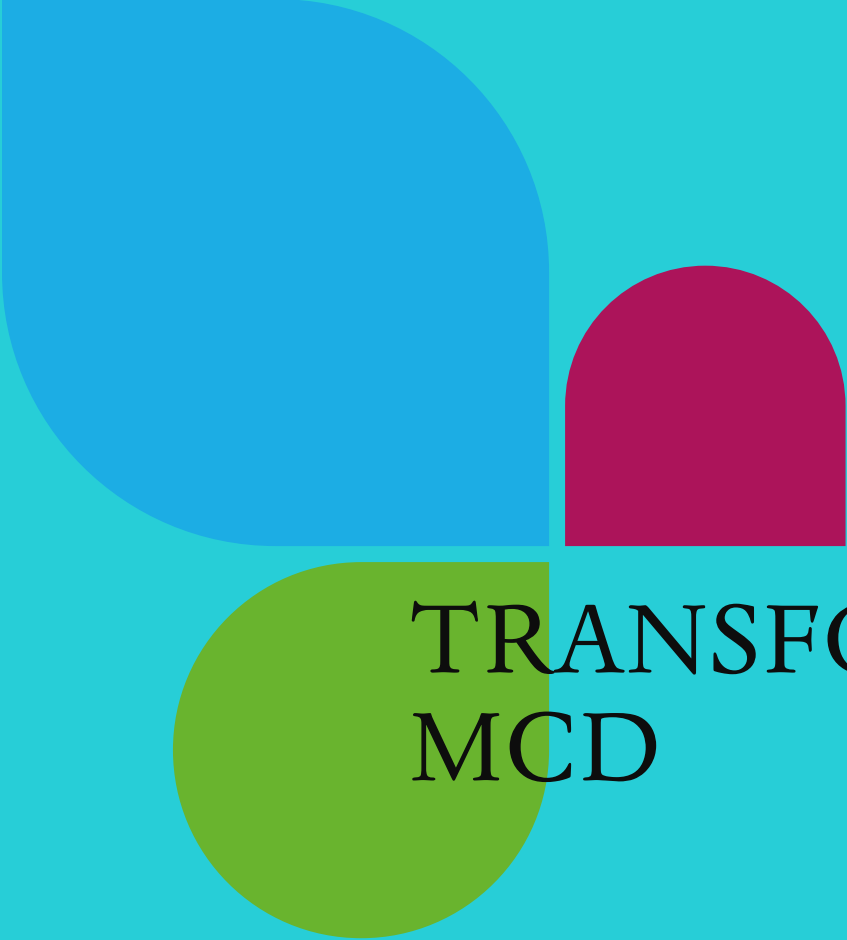
Rôle : Garantit la cohérence référentielle entre entités liées par des associations.

Lien avec Merise : Les CIM sont appliquées aux associations pour gérer les effets de bord, mais ne sont pas des contraintes d'association au sens de la modélisation conceptuelle.

Types de CIM :

Il existe 4 stratégies principales pour gérer les mises à jour, applicables en SQL via la clause ON DELETE ou ON UPDATE :

Stratégie	Description	Risque / Avantage
CASCADE	Propager la modification/suppression aux données dépendantes.	Risque : Perte de données en cascade.
SET NULL	Mettre à NULL la clé étrangère des données dépendantes.	Nécessite que la clé étrangère accepte NULL.
SET DEFAULT	Remplacer la clé étrangère par une valeur par défaut.	Utile pour des valeurs génériques.
RESTRICT (ou NO ACTION)	Bloquer la modification/suppression si des données dépendantes existent.	Sécurité maximale, mais rigidité.

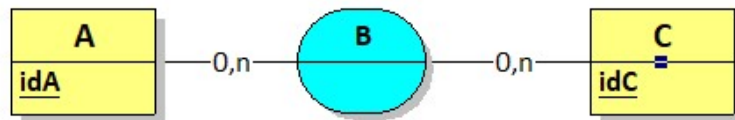


TRANSFORMATION DE MCD

MCD VERS MPD

Relation x,n / x,n

Solution

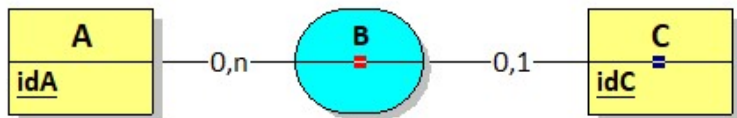
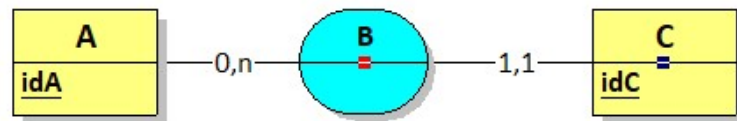


```
A = (idA VARCHAR(50));  
C = (idC VARCHAR(50));  
B = (#idA, #idC);
```

MCD VERS MPD

Relation x,x / x,1

Solution

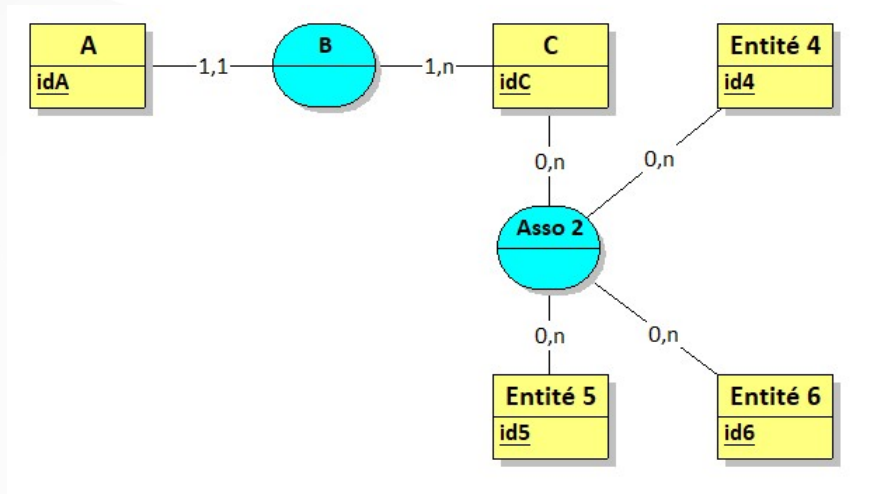


```

A = (idA VARCHAR(50));
C = (idC VARCHAR(50), #idA*);
    
```

CAS AUTO ECOLE

MCD



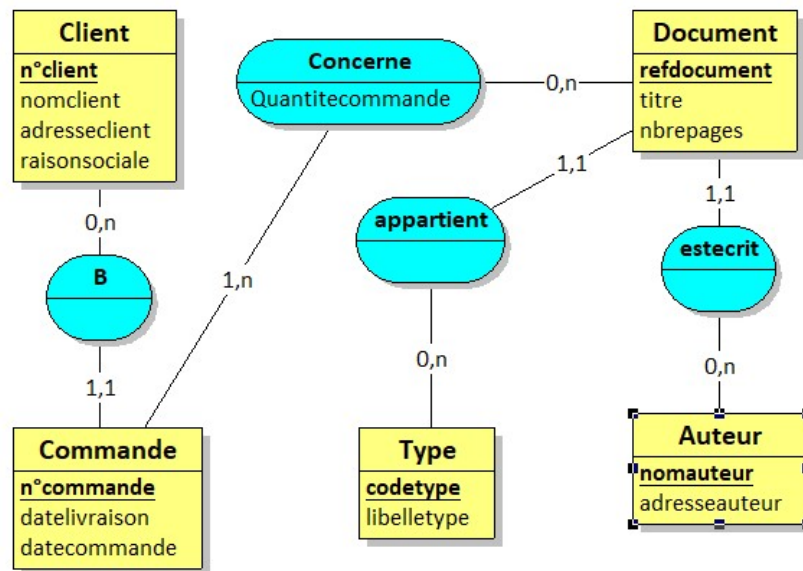
Solution

```

C = (idC VARCHAR(50));
Entité_4 = (id4 VARCHAR(50));
Entité_5 = (id5 VARCHAR(50));
Entité_6 = (id6 VARCHAR(50));
A = (idA VARCHAR(50), #idC);
Asso_2 = (#idC, #id4, #id5, #id6);
  
```


CAS LIBRAIRIE

MCD



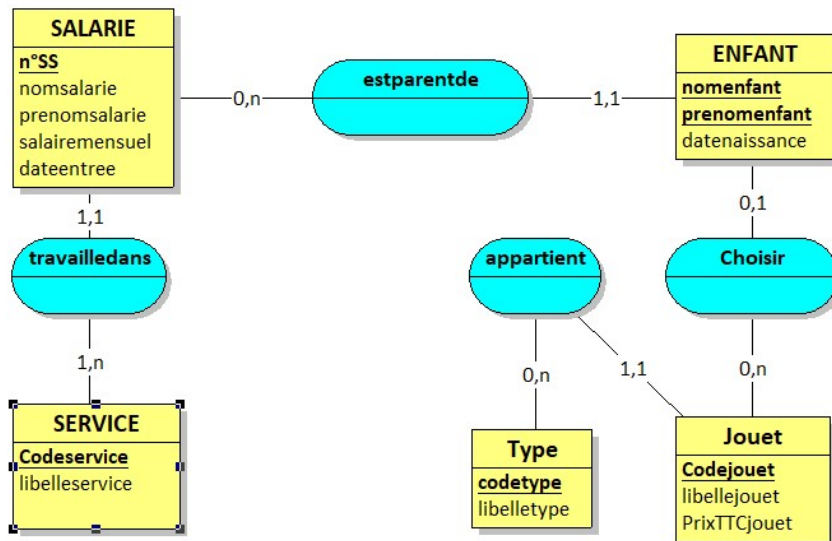
Solution de la transformation

```

Client = (n_client VARCHAR(50), nomclient VARCHAR(50), adresseclient VARCHAR(50), raisonsociale VARCHAR(50));
Commande = (n_commande VARCHAR(50), datelivraison VARCHAR(50), datecommande VARCHAR(50), #n_client);
Type = (codetype VARCHAR(50), libelletype VARCHAR(50));
Auteur = (nomauteur VARCHAR(50), adresseauteur VARCHAR(50));
Document = (refdocument VARCHAR(50), titre VARCHAR(50), nbrepages VARCHAR(50), #codetype, #nomauteur);
Concerne = (#n_commande, #refdocument, Quantitecommande VARCHAR(50));
    
```

COMITÉ D'ENTREPRISE

MCD



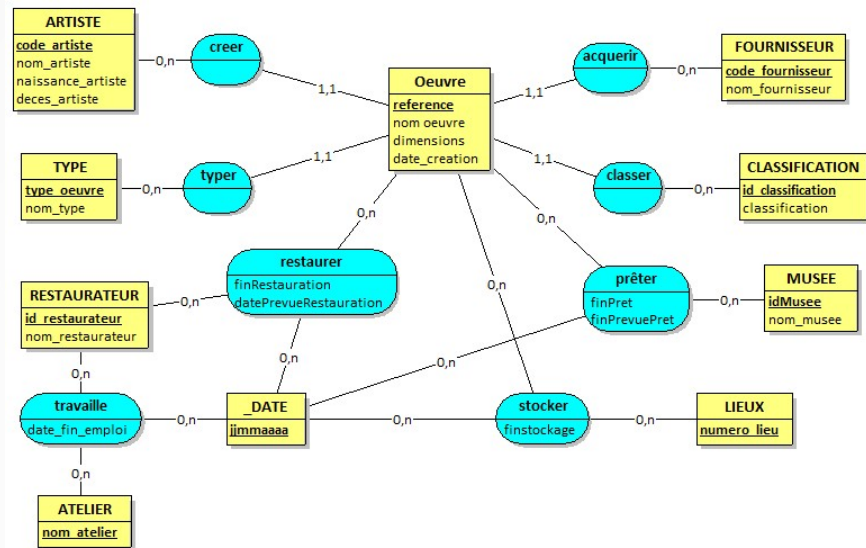
Solution transformation

```

SERVICE = (Codeservice VARCHAR(50), libelleservice VARCHAR(50));
Type = (codetype VARCHAR(50), libelletype VARCHAR(50));
Jouet = (Codejouet VARCHAR(50), libellejouet VARCHAR(50), PrixTTCjouet VARCHAR(50), #codetype);
SALARIE = (n_SS VARCHAR(50), nomsalarie VARCHAR(50), prenomsalarie VARCHAR(50), salairemensuel VARCHAR(50), dateentree VARCHAR(50), #Codeservice);
ENFANT = (nomenfant VARCHAR(50), prenomenfant VARCHAR(50), datenaissance VARCHAR(50), #Codejouet*, #n_SS);
    
```

CAS MUSEE

MCD



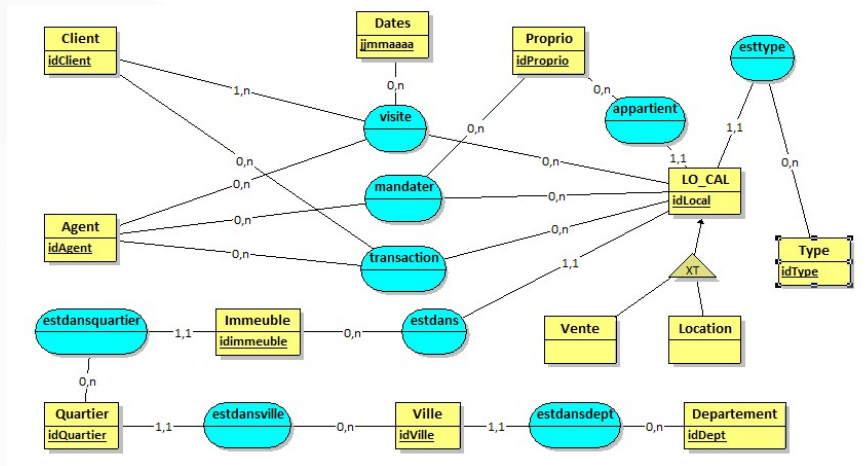
Solution

```

ARTISTE = (code_artiste VARCHAR(50), nom_artiste VARCHAR(50), naissance_artiste DATE, deces_artiste DATE);
FOURNISSEUR = (code_fournisseur VARCHAR(50), nom_fournisseur VARCHAR(50));
TYPE = (type_oeuvre VARCHAR(50), nom_type VARCHAR(50));
CLASSIFICATION = (id_classification VARCHAR(50), classification VARCHAR(50));
RESTAURATEUR = (id_restaurateur VARCHAR(50), nom_restaurateur VARCHAR(50));
_DATE = (jimmaaaa DATE);
MUSEE = (id_musee VARCHAR(50), nom_musee VARCHAR(50));
LIEUX = (numero_lieu VARCHAR(50));
ATELIER = (nom_atelier VARCHAR(50));
Oeuvre = (reference VARCHAR(50), nom_oeuvre VARCHAR(50), dimensions VARCHAR(50), date_creation DATE, #code_artiste, #code_fournisseur, #type_oeuvre, #id_classification);
prêter = (#reference, #jimmaaaa, #id_musee, finPret DATE, finPrevuePret DATE);
restaurer = (#reference, #jimmaaaa, #id_restaurateur, finRestauration DATE, datePrevueRestauration DATE);
stocker = (#reference, #jimmaaaa, #numero_lieu, finstockage DATE);
travaille = (#id_restaurateur, #jimmaaaa, #nom_atelier, date_fin_emploi DATE);
    
```

CAS GESTION IMMOBILIERE

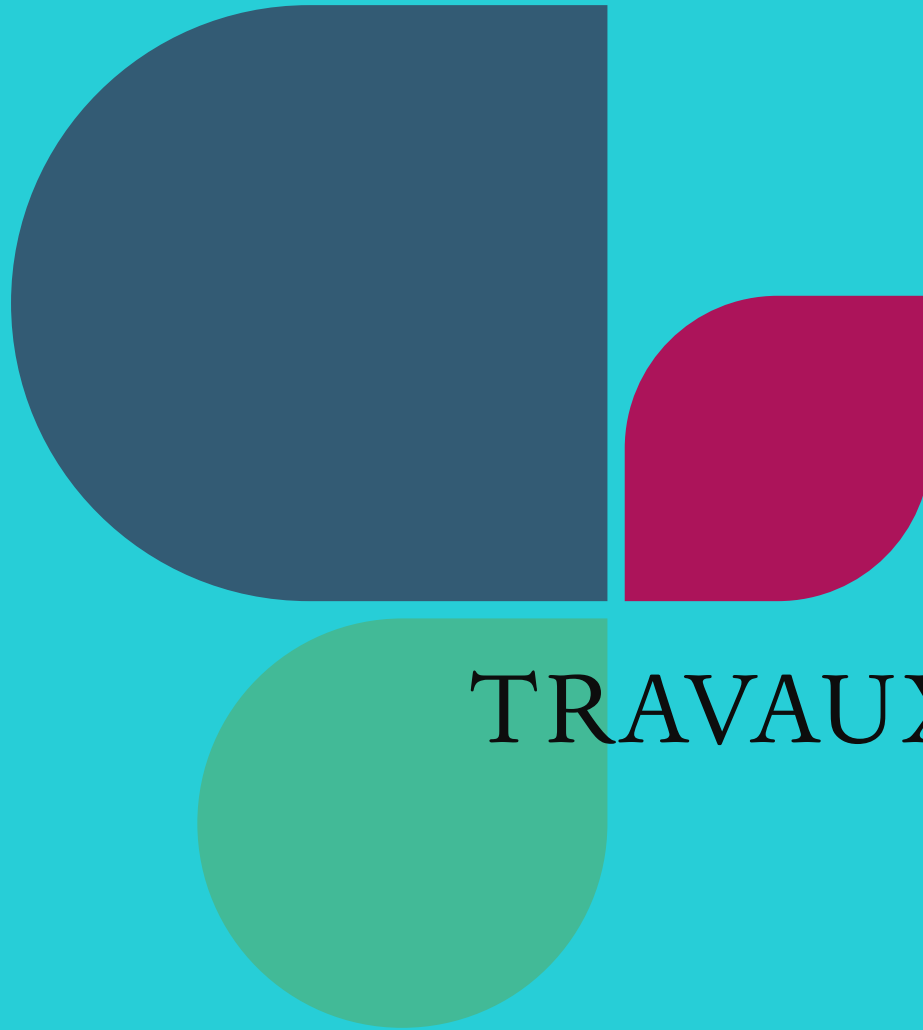
MCD



Solution

```

Departement = (idDept VARCHAR(50));
Ville = (idVille VARCHAR(50), #idDept);
Quartier = (idQuartier VARCHAR(50), #idVille);
Immeuble = (idimmeuble VARCHAR(50), #idQuartier);
Type = (idType VARCHAR(50));
Client = (idClient VARCHAR(50));
Dates = (jjmmaaaa DATE);
Proprio = (idProprio VARCHAR(50));
Agent = (idAgent VARCHAR(50));
LO_CAL = (idLocal VARCHAR(50), #idProprio, #idType, #idimmeuble);
Location = (#idLocal);
Vente = (#idLocal);
transaction = (#idLocal, #idClient, #idAgent);
mandater = (#idLocal, #idProprio, #idAgent);
visite = (#idLocal, #idClient, #jjmmaaaa, #idAgent);
  
```



TRAVAUX PRATIQUES

RÉALISATION UN DES 2 CAS D'ÉTUDES AU CHOIX

Semaine du 29 septembre au 3 octobre

Soit Étudier et réaliser le MCD – MPD du Cas "RACE CHEVALINE"

- A rendre sous forme d'un rapport :
 - Plan
 - Dictionnaires des données
 - MCD
 - MDP

A rendre le 3 octobre - 12h

Soit Étudier et réaliser le MCD – MPD du Cas "JO"

- A rendre sous forme d'un rapport :
 - Plan
 - Dictionnaires des données
 - MCD
 - MDP



FIN...SQL

afpa.fr



Jérôme BOEBION

Version 4 - révision septembre 2025